

SASD Math Toolkit

Numerical Methods - C# / .NET

User and Developer Handbook - Version 1.0 RC1

Numerical methods for robust, inspectable and reusable C# applications.

Repository: github.com/Robin-Goerlach/SASD-Math-Toolkit | C#/.NET
10

Document status

Handbook Version 1.0 RC1

This document is the English release-candidate handbook for SASD Math Toolkit package candidate 1.0.0-rc.1. During the build, chapter content is generated from the maintained Markdown handbook under docs/user-guide/en/. Markdown remains the editorial source of truth while LaTeX provides typography, navigation and PDF output.

Document	SASD Math Toolkit - C#/.NET User and Developer Handbook
Handbook version	1.0 RC1
Package candidate	1.0.0-rc.1
Repository	github.com/Robin-Goerlach/SASD-Math-Toolkit
Target platform	.NET 10
Assembly	Sasd.Math.Toolkit
Root namespace	Sasd.Numerics
License	MIT
Author / project	Robin Goerlach; SASD-GmbH

Preface

SASD Math Toolkit carries the functional core of a classical numerical-methods toolbox into a modern, independently developed C#/.NET library and then extends that foundation for current 2026 workloads. The historical manual is used as orientation for problem classes and handbook structure, not as a source-text or source-code template.

Chapters 1–11 describe the classical numerical foundation. Chapters 12 and 13 document the modern dense and sparse linear-algebra layers added during the 2026 modernization: Householder QR, Cholesky, SVD, conditioning diagnostics, canonical CSR, preconditioning, CG/PCG, restarted GMRES and BiCGSTAB.

Guiding principle

A number alone is not yet a trustworthy numerical result. For important computations, method, tolerance, termination status and a problem-specific quality check belong together.

Contents

Document status	iii
Preface	v
1 Getting started with C#/.NET	1
1.1 Current development form	1
1.2 Namespace organization	1
1.3 A first calculation	1
1.4 Run the graphical numerical demo	1
1.5 Error handling	2
2 Roots of equations	3
2.1 Choosing a method	3
2.2 Common options and result information	3
2.3 Bisection: the bracketed real method	4
2.4 Newton-Raphson: fast when the local model is good	4
2.5 Secant method: derivative-free local iteration	5
2.6 Polynomial coefficients and Horner evaluation	5
2.7 Muller: allowing a real start to become complex	6
2.8 Laguerre for polynomial roots	6
2.9 Interpreting convergence carefully	7
2.10 Practical workflow	7
3 Interpolation	9
3.1 Choosing a method	9
3.2 Lagrange interpolation	9
3.3 Newton divided differences	10
3.4 Natural cubic spline	10
3.5 Clamped cubic spline	10
3.6 Spline domain and immutability	11
3.7 Numerical failure and invalid data	11
3.8 Interpolation is not approximation	12
4 Numerical differentiation	13

4.1	Differentiating a function	13
4.2	Differentiating table data	13
4.3	Differentiating a spline	14
4.4	Choosing a method	14
5	Numerical integration	15
5.1	Choosing a method	15
5.2	Composite trapezoid and Simpson rules	15
5.3	Adaptive Simpson with diagnostics	16
5.4	Romberg integration	16
5.5	Five-point Gauss-Legendre quadrature	17
5.6	Non-finite values and singular integrands	17
5.7	Accuracy is not just a tolerance number	18
6	Matrices and linear systems	19
6.1	1. DenseMatrix	19
6.2	2. Gaussian elimination	20
6.2.1	Partial pivoting	20
6.3	3. Reusable LU factorization	20
6.4	4. Determinants	21
6.5	5. Matrix inverse	21
6.6	6. Pivot tolerance and numerical singularity	21
6.7	7. Residuals	22
6.8	8. Gauss-Seidel iteration	22
6.9	9. Exceptions versus iterative status	23
6.10	10. Method selection	23
6.11	11. Numerical limitations	23
7	Eigenvalues and eigenvectors	25
7.1	Choosing a method	25
7.2	A first power-method example	25
7.3	Eigenvectors are not unique	26
7.4	Checking an eigenpair independently	26
7.5	Power method in more detail	27
7.6	Inverse power method	27
7.7	Wielandt deflation and the second eigenpair	28
7.8	Complete symmetric eigensystem with cyclic Jacobi	28
7.8.1	What does Jacobi's Residual mean?	29

7.9 Tolerances are algorithm criteria, not guaranteed digits	29
7.10 Scope and performance	29
8 Ordinary differential equations and boundary-value problems	31
8.1 First-order initial-value problems	31
8.2 Fixed-step RK4	31
8.3 Second-order equations with RK4	31
8.4 Nth-order equations with RK4	32
8.5 Coupled second-order systems with RK4	33
8.6 Linear boundary-value problems with shooting	33
8.7 Nonlinear boundary-value problems with shooting	34
8.8 Adaptive RKF45	35
8.9 Adams predictor-corrector	35
8.9.1 Why maximumStep is not always the actual step	36
8.10 Choosing between the methods	36
8.11 Boundary residual versus global accuracy	36
8.12 Current limits	37
9 Least-squares approximation	39
9.1 Polynomial fitting	39
9.2 Power-law fitting	39
9.3 Exponential fitting	39
9.4 Logarithmic fitting	39
9.5 Five-term Fourier fitting	40
9.6 Arbitrary linear basis functions	40
9.7 Why QR remains the preferred normal case	40
9.8 When SVD takes over	40
9.9 Understanding residual diagnostics and transformations	41
9.10 Practical checks	41
10 FFT, convolution and correlation	43
10.1 Complex FFT	43
10.2 Real input: full or compact spectrum	43
10.3 Reconstructing the real signal	44
10.4 FFT versus Fourier least-squares approximation	44
10.5 Linear convolution	44
10.6 Cross-correlation and lag ordering	44
10.7 Empty and invalid inputs	45

10.8	Practical limits	45
11	Numerical recipes, diagnostics and common failure modes	47
11.11.	Four questions before accepting a numerical result	47
11.22.	Exceptions and result statuses mean different things	47
11.33.	Converged is a termination statement, not a proof of exactness	48
11.44.	Residual, forward error and estimated error are not interchangeable	48
11.4.1	Root finding	48
11.4.2	Linear systems	49
11.4.3	Eigenpairs	49
11.4.4	Adaptive integration	49
11.4.5	Adaptive ODE integration	49
11.55.	Tolerance must be interpreted in the scale of the problem	50
11.66.	Common termination statuses and what to do next	50
11.6.1	MaximumIterationsReached	50
11.6.2	NumericalBreakdown	50
11.6.3	NotBracketed	50
11.6.4	MaximumDepthReached / NumericalResolutionReached	51
11.6.5	MinimumStepSizeReached	51
11.6.6	Singular or numerically singular matrix	51
11.77.	Non-finite values are treated as failures, not ordinary data	51
11.88.	Conditioning is different from algorithmic convergence	51
11.99.	Independent verification recipes	52
11.9.1	Root	52
11.9.2	Direct linear solve	52
11.9.3	Eigenpair	52
11.9.4	Integration	52
11.9.5	ODE	52
11.9.6	Least squares	52
11.100.	A practical acceptance pattern	53
11.111.	What should be logged	53
11.122.	Compact checklist	53
12	Modern dense linear algebra — 2026 additions	55
12.1	Householder QR	55
12.2	Cholesky	55
12.3	Singular value decomposition	55
12.3.1	Numerical rank is tolerance-dependent	56

12.3.2 Rank-deficient least squares	56
12.3.3 Pseudoinverse	56
12.3.4 Condition number	57
12.4 Matrix norms and a combined diagnostic report	57
12.5 Least-squares routing in the toolkit	58
12.6 Method selection	58
12.7 Performance perspective	58
13 Sparse linear algebra — CSR, CG/PCG, GMRES and BiCGSTAB	59
13.1 Creating a sparse matrix	59
13.2 Matrix-vector multiplication	59
13.3 Dense conversion and sparsification	60
13.4 Transpose	60
13.5 Sparse norms	60
13.6 Conjugate Gradient for SPD systems	60
13.6.1 True residual verification	61
13.7 Preconditioned Conjugate Gradient	61
13.8 Restarted GMRES for general square systems	62
13.8.1 Right-preconditioned GMRES	62
13.9 BiCGSTAB for bounded-memory nonsymmetric solves	62
13.9.1 Right-preconditioned BiCGSTAB	63
13.9.2 Choosing between the sparse solvers	63
13.9.3 Solver diagnostics	63
13.10 What sparse linear algebra does not yet provide	64
A Historical context and current V1 coverage	65
B Building and maintaining the LaTeX edition	67
License and provenance	69

Getting started with C#/.NET

1.1 Current development form

SASD Math Toolkit currently targets .NET 10 and is developed as a class library inside this repository. Until a package feed is introduced, the simplest development setup is to reference `src/dotnet/Sasd.Math.Toolkit/Sasd.Math.Toolkit.csproj` from another project or work directly with the repository solution.

Build the complete repository with:

```
dotnet restore Sasd.Math.Toolkit.slnx
dotnet build Sasd.Math.Toolkit.slnx --configuration Release
dotnet test Sasd.Math.Toolkit.slnx --configuration Release
```

1.2 Namespace organization

The assembly is named `Sasd.Math.Toolkit`, while numerical APIs use the `Sasd.Numerics` namespace root. This avoids ambiguity with `System.Math` and keeps the mathematical domains explicit, for example:

```
using Sasd.Numerics.Differentiation;
using Sasd.Numerics.Integration;
using Sasd.Numerics.LinearAlgebra;
```

1.3 A first calculation

For a callable function, a first derivative can be estimated directly:

```
var derivative = NumericalDifferentiation.FirstDerivative(Math.Sin, 0.3);
Console.WriteLine(derivative);
```

The result should be close to `Math.Cos(0.3)`. Numerical methods return approximations, so comparisons should normally use tolerances rather than exact floating-point equality.

1.4 Run the graphical numerical demo

The repository contains a cross-platform sample that exercises real toolkit APIs and generates a self-contained HTML/SVG report:

```
dotnet run --project samples/dotnet/Sasd.Math.Toolkit.Sample/Sasd.Math.Toolkit.  
Sample.csproj
```

Open the generated `sasd-math-toolkit-demo.html` file in a browser. The report currently includes root finding, integration, compact real FFT analysis, convolution and correlation. It has no network or JavaScript dependency, so it is also useful as a deterministic smoke/demo artifact.

This sample replaces the role of the historical graphics demonstrations with newly written C#/.NET code. Rendering remains outside the numerical library so application projects are not forced to depend on a particular UI or charting technology.

1.5 Error handling

Programming-contract errors such as incompatible dimensions, invalid indices or non-finite inputs are generally reported through standard .NET exceptions. Iterative algorithms whose convergence can legitimately fail usually return a result object containing status, iteration count and residual information.

This distinction is intentional: malformed input is different from a mathematically valid iteration that simply does not converge within the requested limits.

Roots of equations

Root finding asks for a value x for which

$$f(x) = 0.$$

The SASD Math Toolkit separates three related situations: scalar real equations, complex-valued equations, and polynomial roots. The public APIs deliberately expose convergence status instead of pretending that every mathematically valid iteration must succeed.

2.1 Choosing a method

Situation	Recommended starting p
A real root is known to lie inside a sign-changing interval	RootSolvers.Bisection
A good initial guess and derivative are available	RootSolvers.NewtonRaphson
A derivative is unavailable, but two useful guesses are available	RootSolvers.Secant
A polynomial root is wanted and a useful start is known	PolynomialRootSolvers.New
A general complex function should be allowed to leave the real axis	ComplexRootSolvers.Mulle
One polynomial root should be found robustly in the complex plane	PolynomialRootSolvers.Lag
All polynomial roots are needed	PolynomialRootSolvers.Fin

No single root solver is universally best. A bracketed real problem is fundamentally different from finding every complex root of a polynomial.

2.2 Common options and result information

The scalar, Muller, Newton-Horner and Laguerre solvers use `RootFindingOptions`:

```
using Sasd.Numerics.RootFinding;

var options = new RootFindingOptions(
    Tolerance: 1e-12,
    MaximumIterations: 100);
```

Tolerance is positive and is used by the current implementation both for the residual and for a relative-scaled change in the iterate. `MaximumIterations` is a safety limit, not a promise that convergence will occur before that point.

Scalar methods return `RootResult`. The most useful properties are:

- `Root`: the best root estimate returned by the algorithm;

- FunctionValue: $f(\text{Root})$ for that estimate;
- Residual: $\text{abs}(\text{FunctionValue})$;
- Iterations: completed iteration count;
- Status: why the algorithm stopped;
- Converged: shorthand for `Status == IterationStatus.Converged`;
- Message: optional diagnostic text.

Complex single-root methods return `ComplexRootResult`, whose `Residual` is $|f(\text{root})|$. `FindAllRootsLaguerre` returns `PolynomialRootsResult` with the roots found, total iteration count and the largest residual measured against the original polynomial.

Programming-contract errors such as NaN input or an invalid interval are exceptions. Expected numerical outcomes such as a missing bisection bracket, a nearly zero Newton derivative, or reaching the iteration limit are represented by the result status.

2.3 Bisection: the bracketed real method

Bisection is the first choice when a continuous function is known to change sign over an interval. For

$$f(x) = \cos(x) - x$$

the interval $[0, 1]$ brackets the root:

```
using Sasd.Numerics.RootFinding;

var result = RootSolvers.Bisection(
    x => Math.Cos(x) - x,
    left: 0.0,
    right: 1.0);

if (!result.Converged)
{
    Console.WriteLine($"Bisection stopped with {result.Status}: {result.Message}");
}
else
{
    Console.WriteLine($"root = {result.Root:G17}");
    Console.WriteLine($"residual = {result.Residual:E3}");
}
```

Bisection repeatedly halves the bracket and keeps the half whose endpoints still have opposite signs. This gives predictable progress and makes it much less dependent on a clever starting guess than Newton or secant iteration.

If the endpoint values have the same sign, the toolkit returns `IterationStatus.NotBracketed`; it does not invent a root or silently switch algorithms. A same-sign interval does not prove that no root exists inside it, but it does mean that ordinary bisection has no valid sign-change bracket.

2.4 Newton-Raphson: fast when the local model is good

Newton-Raphson uses the tangent line

$x(\text{next}) = x - f(x) / f'(x).$

For the same equation:

```
var result = RootSolvers.NewtonRaphson(
    x => Math.Cos(x) - x,
    x => -Math.Sin(x) - 1.0,
    initialGuess: 0.5);
```

Newton iteration is often much faster than bisection near a simple root, but it relies on local derivative information. A poor initial guess can move the sequence away from the desired root, and a derivative close to zero makes the Newton step unstable. The SASD implementation reports that condition as `IterationStatus.NumericalBreakdown` rather than dividing by an almost-zero value.

Use Newton-Raphson when the derivative is reliable and a plausible initial guess is available. If guaranteed bracket retention matters more than speed, prefer bisection or build a later safeguarded hybrid method on top of the existing primitives.

2.5 Secant method: derivative-free local iteration

The secant method approximates the derivative from two recent function values:

```
var result = RootSolvers.Secant(
    x => Math.Cos(x) - x,
    firstGuess: 0.0,
    secondGuess: 1.0);
```

It needs no derivative callback and commonly converges faster than bisection. Unlike bisection, however, the two guesses do not define a permanently protected bracket. If the two function values become almost equal, the secant slope becomes numerically unusable and the result reports `NumericalBreakdown`.

2.6 Polynomial coefficients and Horner evaluation

Polynomial APIs use `Sasd.Numerics.Polynomials.Polynomial`. Coefficients are supplied in **descending power order**. For example:

```
using Sasd.Numerics.Polynomials;

// x^3 - 2x - 5
var polynomial = new Polynomial([1.0, 0.0, -2.0, -5.0]);
```

This ordering is important. `[1, 0, -2, -5]` does not mean constant-first coefficients.

`PolynomialRootSolvers.NewtonHorner` combines Newton iteration with extended Horner evaluation, so the polynomial value and derivative are obtained efficiently in one pass:

```
using System.Numerics;
using Sasd.Numerics.RootFinding;

var result = PolynomialRootSolvers.NewtonHorner(
    polynomial,
    initialGuess: new Complex(2.0, 0.0));
```

After a root has been found, `Polynomial.Deflate(root)` can remove the corresponding linear factor. The deflation result includes both the quotient polynomial and the remainder. A large remainder is a warning that the supplied value was not an accurate root.

2.7 Muller: allowing a real start to become complex

Muller's method uses three recent points to fit a quadratic local model. Because the quadratic discriminant is evaluated as a complex number, a sequence that begins on the real axis may naturally move into the complex plane.

For $f(z) = z^2 + 1$:

```
using System.Numerics;

static Complex Function(Complex z) => (z * z) + Complex.One;

var result = ComplexRootSolvers.Muller(
    Function,
    Complex.Zero,
    Complex.One,
    new Complex(2.0, 0.0));
```

A converged result should be close to either $+i$ or $-i$. Which root is reached is determined by the numerical path; a single-root solver does not promise a particular member of a multi-root set unless the starting configuration makes that behavior sufficiently constrained.

2.8 Laguerre for polynomial roots

Laguerre's method is specialized for polynomials and uses the value plus the first two derivatives. It is useful when roots may be complex:

```
var quartic = new Polynomial([1.0, 0.0, 0.0, 0.0, 1.0]); // x^4 + 1
var oneRoot = PolynomialRootSolvers.Laguerre(quartic, Complex.One);
```

When all roots are required, use the higher-level operation rather than manually writing a deflation loop:

```
var polynomial = new Polynomial([1.0, 0.0, 0.0, 0.0, -1.0]); // x^4 - 1
var allRoots = PolynomialRootSolvers.FindAllRootsLaguerre(polynomial);

if (allRoots.Converged)
{
    foreach (var root in allRoots.Roots)
    {
        Console.WriteLine(root);
    }

    Console.WriteLine($"maximum residual = {allRoots.MaximumResidual:E3}");
}
```

The current all-roots implementation is deterministic. It uses repeated Laguerre searches, synthetic deflation, fallback start points based on a Cauchy root bound, and a final polishing pass against the original polynomial. Polishing is important because rounding errors from successive deflations can otherwise accumulate.

2.9 Interpreting convergence carefully

A small numerical step and a small residual are related but not identical ideas. The current root solvers accept convergence when either the function residual is within the requested tolerance or the change between successive iterates is sufficiently small relative to scale. For important calculations, inspect `Residual` as well as `Converged`.

A converged root is also not automatically the root you intended. Functions can have multiple roots, repeated roots can slow local methods, and starting guesses influence Newton, secant, Muller and Laguerre iterations. Bisection provides stronger localization because the root remains inside a sign-changing interval, but only for a valid real bracket.

2.10 Practical workflow

For a new problem, first decide whether it is a general real equation, a complex equation, or a polynomial. If a real sign-changing interval is available, begin with bisection as a reference result. Use Newton or secant when speed matters and suitable starting information exists. For polynomials, prefer the polynomial-specific methods because they reuse Horner evaluation and provide deflation/all-roots support. Always record the termination status, residual and tolerances when numerical results are important to later analysis.

Interpolation

Interpolation constructs a function that passes through known data points. It is useful when values are known only at discrete locations but an estimate between them is required.

SASD Math Toolkit V1 provides four closely related choices:

- direct Lagrange polynomial interpolation,
- Newton divided-difference interpolation,
- natural cubic splines,
- clamped cubic splines.

All are interpolation methods: at the supplied knots they reproduce the supplied values, subject only to floating-point round-off. They are not regression methods and do not smooth measurement noise.

3.1 Choosing a method

Use **Lagrange** for a small data set and one or a few evaluations when the direct formula is convenient. Use **Newton divided differences** when the same interpolation polynomial will conceptually be reused or when its coefficient form is useful. Use a **cubic spline** when many knots are present and a piecewise-smooth curve is preferable to one high-degree global polynomial.

Natural and clamped splines differ only in their endpoint conditions. A natural spline assumes zero second derivative at both ends. A clamped spline accepts known first derivatives at both ends.

High-degree global polynomial interpolation can oscillate strongly, especially near the interval boundaries and for unfortunate knot distributions. More data points do not automatically make a global interpolation polynomial better.

3.2 Lagrange interpolation

Suppose the points are (0,1), (1,4) and (2,9). They lie on $(x+1)^2$.

```
using Sasd.Numerics.Interpolation;

double[] x = [0.0, 1.0, 2.0];
double[] y = [1.0, 4.0, 9.0];

var value = InterpolationAlgorithms.Lagrange(x, y, 1.5);
Console.WriteLine(value); // approximately 6.25
```

The x-values must be distinct, but they do not have to be sorted for polynomial

interpolation. Inputs and the evaluation point must be finite.

Lagrange evaluates the basis formula directly. This keeps the implementation easy to inspect but repeats the $O(n^2)$ basis work for every call. V1 deliberately favors this clear reference implementation over premature optimization.

3.3 *Newton divided differences*

The same interpolation polynomial can be represented as

$$a_0 + a_1(x-x_0) + a_2(x-x_0)(x-x_1) + \dots$$

The coefficients can be obtained separately:

```
var coefficients = InterpolationAlgorithms.NewtonDividedDifferenceCoefficients(x, y);
```

For the example above they are approximately [1, 3, 1] in the Newton basis defined by the supplied x-order.

For a one-off evaluation there is a convenience method:

```
var value = InterpolationAlgorithms.NewtonDividedDifference(x, y, 1.5);
```

The returned coefficient array is independent of the input arrays. Changing the input after coefficient construction does not change the coefficients.

3.4 *Natural cubic spline*

A cubic spline uses a separate cubic polynomial on every interval while enforcing continuity of the function and its first two derivatives at interior knots.

```
double[] x = [0.0, 1.0, 2.0, 3.0];  
double[] y = [0.0, 1.0, 0.0, 1.0];  
  
var spline = InterpolationAlgorithms.NaturalCubicSpline(x, y);  
  
var value = spline.Evaluate(1.5);  
var slope = spline.FirstDerivative(1.5);  
var curvature = spline.SecondDerivative(1.5);
```

A natural spline adds the endpoint conditions

$$S''(x_0) = 0 \text{ and } S''(x_n) = 0.$$

This is a neutral boundary assumption, not a statement that the underlying real function truly has zero curvature there.

3.5 *Clamped cubic spline*

If endpoint slopes are known, a clamped spline can incorporate them:

```
static double F(double x) => x * x * x;
```

```
static double Df(double x) => 3.0 * x * x;

double[] x = [0.0, 1.0, 2.0, 3.0];
var y = x.Select(F).ToArray();

var spline = InterpolationAlgorithms.ClampedCubicSpline(
    x,
    y,
    leftDerivative: Df(x[0]),
    rightDerivative: Df(x[^1]));
```

With exact endpoint derivatives, a clamped cubic spline reproduces a cubic polynomial exactly apart from rounding error. In measured data, endpoint derivatives should only be supplied when they are meaningfully known; inaccurate slope constraints can distort the boundary region.

3.6 Spline domain and immutability

Spline knots must be strictly increasing. The spline exposes:

```
spline.Knots
spline.IntervalStart
spline.IntervalEnd
spline.SegmentCount
```

Knots is read-only, and the spline owns its internal coefficient data. The object can therefore be safely reused after construction without being changed by later modifications of input arrays.

Spline evaluation deliberately does **not** extrapolate. Calls outside [IntervalStart, IntervalEnd], and calls with NaN or infinity, throw `ArgumentOutOfRangeException`. If extrapolation is required, it should be an explicit modeling decision rather than an accidental side effect of interpolation.

Polynomial Lagrange and `NewtonDividedDifference` do permit evaluation outside the supplied x-range because a global polynomial is mathematically defined there. That does not make polynomial extrapolation reliable.

3.7 Numerical failure and invalid data

Duplicate polynomial abscissas are invalid because the interpolation polynomial would require division by zero. Spline knots must additionally be sorted and strictly increasing. Non-finite input data are rejected.

Finite input can still be so extremely scaled that intermediate floating-point arithmetic overflows or a spline system becomes numerically unusable. In those cases the reference implementation raises `ArithmeticException` rather than silently returning NaN or infinity.

For difficult data sets, useful first checks are to inspect scales, shift or normalize x-values when mathematically appropriate, and avoid unnecessarily high-degree global polynomials.

3.8 Interpolation is not approximation

Interpolation forces the resulting curve through every supplied value. Least-squares approximation, described in the later handbook chapter, intentionally allows residuals in order to fit a model to noisy or overdetermined data. Choose based on the meaning of the data, not merely on which method produces a visually smooth curve.

Numerical differentiation

Numerical differentiation estimates a derivative when an analytic derivative is unavailable or inconvenient. SASD Math Toolkit provides three workflows: differentiate a callable function, differentiate sampled table data, or differentiate a cubic spline fitted to table data.

4.1 Differentiating a function

```
using Sasd.Numerics.Differentiation;  
  
var first = NumericalDifferentiation.FirstDerivative(Math.Sin, 0.3);  
var second = NumericalDifferentiation.SecondDerivative(Math.Sin, 0.3);
```

The default routines use centered differences plus Richardson refinement. Explicit five-point methods are available when that stencil is preferred.

4.2 Differentiating table data

Suppose a measurement series contains increasing x-values and corresponding y-values:

```
var x = new[] { 0.0, 0.5, 1.1, 2.0, 3.0 };  
var y = x.Select(value => value * value).ToArray();  
  
var first = TabularDifferentiation.FirstDerivativeThreePoint(x, y, 2);  
var second = TabularDifferentiation.SecondDerivativeFivePoint(x, y, 2);
```

The final argument is the index of the table entry at which the derivative is required. The x-values may be non-uniformly spaced but must be finite and strictly increasing.

Available methods are:

- first derivative: two, three or five points;
- second derivative: three or five points.

Near the first or last table entry the library automatically shifts the local stencil instead of reading beyond the table. For smooth data, more points often reduce truncation error. For noisy measurements, however, differentiation can amplify noise, and a larger stencil is not automatically better.

4.3 Differentiating a spline

A spline is useful when the data should first be represented by a smooth piecewise cubic curve. For repeated evaluations, build the spline once:

```
using Sasd.Numerics.Interpolation;

var spline = InterpolationAlgorithms.NaturalCubicSpline(x, y);
var slope = spline.FirstDerivative(1.4);
var curvature = spline.SecondDerivative(1.4);
```

For a one-shot calculation, convenience helpers are available:

```
var slope = SplineDifferentiation.NaturalFirstDerivative(x, y, 1.4);
```

If endpoint derivatives are known, use the clamped variant:

```
var slope = SplineDifferentiation.ClampedFirstDerivative(
    x,
    y,
    leftDerivative: 0.0,
    rightDerivative: 6.0,
    point: 1.4);
```

Spline evaluation is restricted to the interpolation interval. SASD Math Toolkit does not silently extrapolate beyond the supplied data.

4.4 Choosing a method

Use direct function differentiation when a function can be evaluated freely near the target point. Use tabular differentiation when the original data points themselves are the object of interest. Use spline differentiation when a smooth interpolating curve is a meaningful model for the data and derivatives are needed at arbitrary positions inside the interval.

No method can recover information that is absent from the data. In particular, derivative estimates from noisy measurements should be interpreted with more caution than derivatives of smooth mathematical functions.

Numerical integration

Numerical integration approximates a definite integral when an antiderivative is unavailable, inconvenient, or represented only by executable code. SASD Math Toolkit provides fixed-panel Newton-Cotes rules, Romberg extrapolation, five-point Gauss-Legendre quadrature, and adaptive variants.

```
using Sasd.Numerics.Integration;
```

All current one-dimensional routines require finite bounds with $a < b$. If the mathematical integral is needed in the opposite direction, swap the endpoints and negate the result explicitly.

5.1 Choosing a method

Use **composite trapezoid** when a simple baseline method is useful or the data model naturally suggests piecewise linear behavior. Use **composite Simpson** for smooth functions when an even fixed panel count is acceptable. Use **adaptive Simpson** when local difficulty varies strongly across the interval and you want refinement to concentrate where it is needed.

Use **Romberg** for smooth functions when repeated trapezoid refinement and Richardson extrapolation are effective. Use **five-point Gauss-Legendre** when one high-order panel is appropriate; the rule integrates polynomials through degree nine exactly in exact arithmetic. Use **adaptive five-point Gauss-Legendre** when the fixed rule is attractive but one panel is not sufficient over the whole interval.

5.2 Composite trapezoid and Simpson rules

```
var trapezoid = NumericalIntegration.CompositeTrapezoid(  
    Math.Sin,  
    0.0,  
    Math.PI,  
    intervals: 200);  
  
var simpson = NumericalIntegration.CompositeSimpson(  
    Math.Sin,  
    0.0,  
    Math.PI,  
    intervals: 200);
```

The panel count is part of the numerical model. More panels usually reduce discretization error for a sufficiently smooth function, but they also increase callback evaluations and eventually expose floating-point limitations. Composite Simpson requires an even

number of intervals.

For low-degree polynomials, the exactness properties are useful sanity checks: the trapezoid rule is exact for linear functions, while Simpson's rule is exact for cubic polynomials in exact arithmetic.

5.3 *Adaptive Simpson with diagnostics*

The simple API returns only the best estimate:

```
var integral = NumericalIntegration.AdaptiveSimpson(  
    Math.Sin,  
    0.0,  
    Math.PI,  
    tolerance: 1e-10,  
    maximumDepth: 20);
```

For production code that needs to distinguish a successful tolerance test from a forced stop, prefer the detailed API:

```
var result = NumericalIntegration.AdaptiveSimpsonDetailed(  
    Math.Sin,  
    0.0,  
    Math.PI,  
    tolerance: 1e-10,  
    maximumDepth: 20);  
  
if (!result.Converged)  
{  
    Console.WriteLine($"Stopped with {result.Status}");  
}  
  
Console.WriteLine(result.Value);  
Console.WriteLine(result.EstimatedError);  
Console.WriteLine(result.FunctionEvaluations);
```

`AdaptiveIntegrationStatus.Converged` means every accepted panel met its local criterion. `MaximumDepthReached` means at least one region still wanted refinement when the configured depth was exhausted. `NumericalResolutionReached` means the interval could no longer be bisected into distinct double values. In either non-converged case, `Value` remains the best estimate available to the algorithm, but it should not be presented as if the requested tolerance had been guaranteed.

The Simpson error indicator is based on the difference between one Simpson panel and its two half-panels, with the usual Richardson correction. It is an estimate, not a proof of the true mathematical error.

5.4 *Romberg integration*

Romberg integration builds a triangular extrapolation table from successively halved trapezoid rules:

```
var result = NumericalIntegration.RombergDetailed(  
    x => Math.Exp(-x * x),
```

```
0.0,
1.0,
tolerance: 1e-10,
maximumLevels: 12);
```

`result.Value` is the best diagonal extrapolation. `EstimatedError` is the absolute difference between the last two diagonal estimates. `Status` is `IterationStatus.Converged` when that difference meets the requested tolerance; otherwise the last available estimate is returned with `MaximumIterationsReached`.

The work approximately doubles with every additional trapezoid level. The reference implementation therefore limits `maximumLevels` to 30 to keep the integer subdivision model well-defined. Values anywhere near that upper bound can still be computationally expensive; normal applications should use far fewer levels.

5.5 Five-point Gauss-Legendre quadrature

A fixed five-node rule is available directly:

```
var integral = NumericalIntegration.GaussLegendre5(
    x => Math.Exp(x),
    0.0,
    1.0);
```

The nodes and weights are mapped from the standard interval $[-1, 1]$ onto $[a, b]$. For smooth functions, a Gaussian rule can obtain high accuracy with relatively few function evaluations.

The adaptive variant compares one five-node rule over a panel with the sum of two five-node rules over its halves:

```
var result = NumericalIntegration.AdaptiveGaussLegendre5Detailed(
    Math.Exp,
    0.0,
    1.0,
    tolerance: 1e-12,
    maximumDepth: 16);
```

Its `EstimatedError` is the accumulated refinement difference used by this algorithm. As with adaptive Simpson, it is an error indicator rather than a formal bound.

5.6 Non-finite values and singular integrands

All integration routines reject NaN and infinity returned by the integrand. This is intentional: silently propagating a non-finite sample through a long quadrature can produce a result that is difficult to diagnose.

An improper integral or a function with an endpoint singularity therefore needs an explicit transformation or interval treatment by the application. The current V1 routines are finite-interval quadrature methods; they do not automatically interpret infinite limits or principal values.

5.7 Accuracy is not just a tolerance number

A requested tolerance only controls the algorithm's own refinement criterion. It cannot compensate for a badly scaled integrand, discontinuities, unresolved narrow peaks, cancellation, or loss of precision inside the user callback. When a result matters, vary the method or numerical controls and check whether the answer is stable.

The detailed adaptive and Romberg result types exist specifically so applications can record how an answer was obtained instead of storing only a bare double.

Matrices and linear systems

Many numerical methods eventually reduce a problem to a linear system

$$A * x = b$$

where A is a coefficient matrix, x is the unknown vector and b is the right-hand side. SASD Math Toolkit provides a deliberately small dense-matrix foundation plus direct and iterative solvers suitable for the V1 reference/educational scope.

The implementation is intended to be understandable first. It is not a substitute for a large sparse solver or a tuned BLAS/LAPACK stack.

6.1 1. DenseMatrix

Create a matrix from a rectangular C# array:

```
using Sasd.Numerics.LinearAlgebra;

var matrix = new DenseMatrix(new double[,]
{
    { 4.0, 1.0 },
    { 2.0, 3.0 }
});

Console.WriteLine(matrix.Rows);    // 2
Console.WriteLine(matrix.Columns); // 2
Console.WriteLine(matrix.IsSquare); // true
```

Entries are mutable:

```
matrix[0, 1] = 1.5;
```

but dimensions are fixed after construction. Matrix entries must be finite. NaN and infinity are rejected because they otherwise tend to contaminate every later numerical operation.

`Clone()` returns an independent matrix. Matrix-vector and matrix-matrix multiplication are also available:

```
var y = matrix.Multiply(new[] { 1.0, 2.0 });
var product = matrix.Multiply(DenseMatrix.Identity(2));
```

The multiplication routines intentionally use straightforward loops. If arithmetic overflows, they throw instead of returning a matrix containing infinity.

6.2 2. Gaussian elimination

For a one-off dense system, SolveGaussian is the direct textbook solver:

```
var a = new DenseMatrix(new double[,]  
{  
    { 2.0, 1.0 },  
    { 5.0, 7.0 }  
});  
  
var b = new[] { 11.0, 13.0 };  
var x = LinearSystemSolvers.SolveGaussian(a, b);
```

The method copies A and b; caller-owned inputs are not modified.

6.2.1 Partial pivoting

Partial pivoting is enabled by default and should normally remain enabled. At each elimination step the solver chooses the largest available absolute entry in the pivot column.

For example, a nonsingular matrix can start with a zero diagonal element:

```
0 2  
1 3
```

Unpivoted Gaussian elimination cannot use the first diagonal entry. Partial pivoting swaps the rows and proceeds normally.

The API retains `partialPivoting: false` for historical and educational comparison, not because it is the recommended production setting.

6.3 3. Reusable LU factorization

If several systems use the same coefficient matrix, do not repeat Gaussian elimination. Factor the matrix once:

```
var lu = LinearSystemSolvers.FactorizeLu(a);  
  
var x1 = lu.Solve(new[] { 11.0, 13.0 });  
var x2 = lu.Solve(new[] { 4.0, 9.0 });
```

The convention is

```
P * A = L * U
```

where P is a row permutation, L is unit-lower-triangular and U is upper-triangular.

For inspection and diagnostics:

```
var lower = lu.LowerTriangular;  
var upper = lu.UpperTriangular;  
var permutation = lu.Permutation;  
var toleranceUsed = lu.PivotTolerance;
```

These exposed matrices/arrays are independent copies where mutation would otherwise compromise the factorization.

Several right-hand sides can be supplied as the columns of one `DenseMatrix`:

```
var bMany = DenseMatrix.Identity(2);
var xMany = lu.Solve(bMany);
```

This is exactly the pattern used when constructing an inverse.

6.4 4. Determinants

A determinant can be computed directly:

```
var determinant = LinearSystemSolvers.Determinant(a);
```

or from an existing LU factorization:

```
var determinant = lu.Determinant();
```

The LU path is preferable when a factorization already exists because it reuses the diagonal of `U` and the row-permutation parity.

A determinant close to zero can indicate singularity, but determinant magnitude by itself is not a robust condition-number estimate. It also changes strongly with matrix scaling.

6.5 5. Matrix inverse

The toolkit exposes:

```
var inverse = LinearSystemSolvers.Inverse(a);
```

Internally this creates one LU factorization and solves against all columns of the identity matrix.

However, if the actual task is to solve $Ax=b$, prefer `SolveGaussian` or `LuFactorization.Solve`. Explicitly forming A^{-1} usually performs more work and can amplify numerical error without adding useful information.

Use the inverse when an algorithm genuinely needs the inverse matrix itself, not merely because the symbolic formula says $x=A^{-1}b$.

6.6 6. Pivot tolerance and numerical singularity

Direct solvers use a positive absolute `pivotTolerance`. A candidate pivot whose magnitude is at or below that threshold is treated as numerically zero.

```
var lu = LuFactorization.Decompose(a, pivotTolerance: 1e-12);
```

This is a practical safety threshold, not a full conditioning analysis. A matrix can have pivots larger than the threshold and still be badly conditioned.

For differently scaled problem families, the same absolute tolerance may not be equally meaningful. V1 deliberately keeps the rule explicit and simple; later SASD work can add scaling and condition estimation without changing the basic solve abstraction.

6.7 7. Residuals

After solving, verify what equation the computed vector actually satisfies:

```
var residual = LinearSystemSolvers.ResidualInfinityNorm(a, x, b);
```

This computes

$$||A*x - b||_\infty = \max_i |(A*x-b)_i|$$

A small residual is useful evidence, but it does not by itself prove that x is close to the exact solution when A is badly conditioned.

This distinction is central:

- **residual** asks how well the computed vector satisfies the supplied equations;
- **forward error** asks how close the vector is to the exact mathematical solution;
- **conditioning** describes how strongly the solution can react to small changes in the input data.

V1 exposes the residual, not a condition estimator.

6.8 8. Gauss-Seidel iteration

Gauss-Seidel is an iterative alternative:

```
var result = LinearSystemSolvers.GaussSeidel(  
    new DenseMatrix(new double[,]  
    {  
        { 4.0, 1.0 },  
        { 2.0, 3.0 }  
    }  
),  
    new[] { 1.0, 2.0 },  
    tolerance: 1e-12,  
    maximumIterations: 200);  
  
if (result.Converged)  
{  
    Console.WriteLine(result.Value[0]);  
    Console.WriteLine(result.Residual);  
}
```

The implementation reports convergence only when both the largest component update and the residual infinity norm are within the requested tolerance.

Possible statuses include:

- **Converged** — the V1 convergence checks passed;
- **MaximumIterationsReached** — the iteration budget was exhausted;

- `NumericalBreakdown` — for example a zero/near-zero diagonal or numerical overflow was encountered.

Gauss-Seidel is **not guaranteed to converge for every nonsingular matrix**. Strict diagonal dominance is a common sufficient condition, though not a necessary one. If you simply need a solution to a small arbitrary dense system, pivoted LU is the safer default.

6.9 9. Exceptions versus iterative status

The linear-algebra API follows the same general SASD convention as the rest of the toolkit.

Invalid contracts such as wrong dimensions, non-finite input values or a non-positive tolerance produce exceptions. A direct factorization that cannot proceed because a usable pivot does not exist also throws `ArithmeticException`.

Gauss-Seidel, on the other hand, is an iterative process. Expected iterative outcomes such as iteration exhaustion or numerical breakdown are represented through `IterativeResult<double[]>` and `IterationStatus`.

6.10 10. Method selection

A practical V1 rule of thumb is:

Situation	Preferred starting point
One small dense system	Pivoted <code>SolveGaussian</code>
Many right-hand sides with the same A	<code>FactorizeLu</code> then repeated <code>Solve</code>
Determinant after an LU already exists	<code>lu.Determinant()</code>
Need the inverse matrix itself	<code>lu.Inverse()</code> / <code>LinearSystemSolvers.Inverse</code>
Iterative experiment on a suitable matrix	<code>GaussSeidel</code>
Large/sparse/high-performance problem	Use a specialized backend rather than the V1 refer

6.11 11. Numerical limitations

Linear systems are one of the clearest examples of why “the program produced a number” is not the same as “the problem is numerically safe.”

Watch for:

- badly scaled rows or columns;
- nearly dependent equations;
- large differences in coefficient magnitude;
- a residual that is small only because the whole problem is scaled very large or very small;
- iterative methods applied without checking their convergence behavior.

The V1 implementation deliberately provides partial pivoting, explicit pivot thresholds,

residuals and failure diagnostics while remaining small enough to audit. Condition estimation, sparse storage and high-performance kernels belong to later milestones rather than being hidden behind an apparently simple V1 API.

Eigenvalues and eigenvectors

Eigenvalue problems ask for a scalar λ and a non-zero vector v such that

$$A * v = \lambda * v.$$

They appear in stability analysis, vibration modes, repeated linear transformations, principal directions and many other numerical models. SASD Math Toolkit V1 contains several classical real dense-matrix methods. They solve different eigenvalue tasks; there is no single routine that should be used for every matrix.

7.1 Choosing a method

Use **PowerMethod** when you need one eigenpair associated with the eigenvalue of largest magnitude and the spectrum has a reasonably clear dominant direction.

Use **InversePowerMethod** when you need the eigenvalue closest to zero and the matrix is nonsingular. The current V1 method is unshifted. It does not secretly choose a spectral shift for you.

Use **WielandtSecondEigenpair** mainly when you specifically want the classical historical workflow for the second-largest eigenvalue magnitude. It is useful as a reference algorithm, but repeated or nearly repeated dominant eigenvalues are a known weak point.

Use **CyclicJacobi** when the matrix is real and symmetric and you want the complete eigensystem. This is generally the most informative V1 API for a small dense symmetric problem because it returns all eigenvalues and an orthonormal eigenvector basis.

V1 does not contain a general nonsymmetric QR/Schur eigensolver or a general complex-eigenvalue API.

7.2 A first power-method example

```
using Sasd.Numerics.LinearAlgebra;

var matrix = new DenseMatrix(new double[,]
{
    { 4.0, 1.0, 0.0 },
    { 1.0, 3.0, 0.0 },
    { 0.0, 0.0, 1.0 }
});

var result = EigenSolvers.PowerMethod(
    matrix,
    tolerance: 1e-12,
    maximumIterations: 500);
```

```
if (result.Converged)
{
    var pair = result.Value;
    Console.WriteLine($"lambda = {pair.Eigenvalue}");
    Console.WriteLine($"residual = {result.Residual}");

    foreach (var component in pair.Components)
    {
        Console.WriteLine(component);
    }
}
else
{
    Console.WriteLine($"{{result.Status}}: {{result.Message}}");
}
```

The result is not just a number. `IterativeResult<Eigenpair>` tells you how the algorithm terminated:

- Converged means the method's convergence tests were satisfied;
- Iterations is the number of iterations used;
- Residual is the Euclidean eigen-equation defect used by the power/inverse-power routines;
- MaximumIterationsReached means the latest approximation is returned but the requested criterion was not reached;
- NumericalBreakdown describes a numerical state in which the iteration could not continue meaningfully.

Invalid method parameters are reported as argument exceptions instead of normal iteration statuses.

7.3 Eigenvectors are not unique

If v is an eigenvector, then every non-zero scalar multiple of it is also an eigenvector. In particular, v and $-v$ are the same eigendirection. SASD's iterative solvers normalize returned vectors, but your tests should normally not fail merely because another mathematically valid implementation chose the opposite sign.

For a repeated eigenvalue the situation is broader: individual eigenvectors inside the repeated eigenspace are not unique. A complete symmetric eigensolver may return a different orthonormal basis of that same subspace and still be correct.

7.4 Checking an eigenpair independently

For a pair (λ, v) , the quantity

$$||A*v - \lambda*v||^2$$

measures how well the pair satisfies the defining equation. SASD exposes this directly:

```
var residual = EigenSolvers.EigenpairResidualNorm(matrix, result.Value);
```

This is an excellent regression and plausibility check. A small residual does **not** mean that the eigenvalue problem is well-conditioned. Closely clustered eigenvalues can be sensitive even while a returned pair has a small equation defect.

Eigenpair itself owns its vector. The public Components view is read-only, while Eigenvector returns a defensive array copy. Modifying that copy cannot mutate the numerical result.

7.5 Power method in more detail

Power iteration repeatedly applies the matrix and normalizes the resulting vector. With suitable assumptions, components belonging to eigenvalues of smaller magnitude become relatively less important and the dominant eigendirection emerges.

A practical limitation follows immediately: the initial vector must have a non-zero component in that eigendirection. SASD uses a deterministic all-ones vector by default. If you know that this vector is orthogonal to the eigenvector you seek, provide another start vector:

```
var result = EigenSolvers.PowerMethod(
    matrix,
    initialVector: new[] { 1.0, 0.2, -0.4 },
    tolerance: 1e-12);
```

A small spectral gap between the largest and second-largest magnitudes usually means slower convergence. If two dominant magnitudes are equal, plain power iteration may be a poor choice.

The implementation uses scaled vector normalization, so very large but finite start vectors do not fail merely because their components would overflow if squared directly.

7.6 Inverse power method

Unshifted inverse iteration repeatedly solves

$$A * y = x$$

and normalizes y . Directions associated with small-magnitude eigenvalues are amplified by the inverse operation. SASD factorizes A once with partial-pivoted LU and reuses that factorization for all iterations.

```
var smallest = EigenSolvers.InversePowerMethod(
    matrix,
    tolerance: 1e-12,
    maximumIterations: 200,
    pivotTolerance: 1e-14);
```

pivotTolerance belongs to the LU setup: pivots at or below that threshold are treated as numerically singular. It should not be confused with the eigenvalue convergence tolerance.

Because the V1 method is unshifted, a singular matrix cannot be processed by this API: the LU factorization fails before the iteration begins. A future shifted inverse iteration will deserve an explicit shift parameter rather than hidden behavior.

7.7 *Wielandt deflation and the second eigenpair*

The classical Wielandt transformation removes one known eigenvalue from a matrix by a rank-one update. The direct API is useful when you already have an approximate eigenpair:

```
var dominant = EigenSolvers.PowerMethod(matrix, tolerance: 1e-12);
if (!dominant.Converged)
{
    throw new InvalidOperationException("Dominant eigenpair did not converge.");
}

var deflation = WielandtDeflation.Create(
    matrix,
    dominant.Value,
    residualTolerance: 1e-8);

Console.WriteLine(deflation.SourceResidual);
```

The source residual is checked because deflating an arbitrary vector/eigenvalue claim would produce a matrix, but not a trustworthy mathematical deflation of the intended eigenvalue.

For the complete historical workflow, use:

```
var second = EigenSolvers.WielandtSecondEigenpair(
    matrix,
    tolerance: 1e-11);
```

This first computes the dominant pair, deflates it, computes the dominant pair of the deflated matrix and restores the second eigenvector in the original coordinates.

Repeated or nearly repeated dominant eigenvalues can make restoration ill-conditioned. Check `second.Status` and `second.Message`; do not treat every returned value as converged.

7.8 *Complete symmetric eigensystem with cyclic Jacobi*

For a real symmetric matrix, the spectral theorem gives real eigenvalues and an orthonormal eigenvector basis. Cyclic Jacobi exploits that structure by repeatedly applying rotations that reduce off-diagonal entries.

```
var decompositionResult = EigenSolvers.CyclicJacobi(
    matrix,
    tolerance: 1e-12,
    maximumSweeps: 100,
    symmetryTolerance: 1e-12);

if (!decompositionResult.Converged)
{
    Console.WriteLine(decompositionResult.Message);
}

var decomposition = decompositionResult.Value;
for (var i = 0; i < decomposition.Size; i++)
```

```

{
    var lambda = decomposition.GetEigenvalue(i);
    var vector = decomposition.GetEigenvector(i);
    var pair = decomposition.GetEigenpair(i);
    var equationResidual = EigenSolvers.EigenpairResidualNorm(matrix, pair);

    Console.WriteLine($"{i}: lambda={lambda}, residual={equationResidual}");
}

```

Important conventions:

- eigenvalues are sorted in descending **numerical value**, not descending absolute magnitude;
- eigenvectors are columns of `decomposition.Eigenvectors`;
- `GetEigenvalue` is convenient when you only need one scalar without allocating an eigenvalue-array copy;
- `GetEigenvector` returns an independent copy;
- the decomposition's public matrix/array properties are also defensive copies.

7.8.1 What does Jacobi's Residual mean?

For `CyclicJacobi`, `decompositionResult.Residual` is the off-diagonal Frobenius norm of the transformed working matrix. It tells you how close the Jacobi process is to diagonal form.

That is a different diagnostic from $\|A*v - \lambda*v\|^2$. If you want to validate the final individual eigenpairs, call `EigenpairResidualNorm` as shown above.

7.9 Tolerances are algorithm criteria, not guaranteed digits

A tolerance such as $1e-12$ is a stopping threshold inside the numerical method. It is not a promise of twelve correct decimal digits. Scaling, conditioning, floating-point roundoff, spectral gaps and the structure of the matrix all matter.

For difficult problems, useful checks include:

1. inspect `Status`, `Iterations` and the algorithm residual;
2. independently evaluate `EigenpairResidualNorm`;
3. for a symmetric complete decomposition, check eigenvector orthogonality when it matters to the application;
4. repeat with a stricter/looser tolerance or another valid start vector when sensitivity is suspected;
5. compare against an independent mature library for high-consequence or large-scale work.

7.10 Scope and performance

These algorithms target small and medium dense reference problems, education, deterministic SASD tests and future cross-language conformance. They deliberately use understandable loops rather than blocked kernels, SIMD-specific implementations

or native dependencies.

For large dense, sparse or high-performance eigenproblems, a future SASD back-end should delegate to mature BLAS/LAPACK or specialized sparse libraries while preserving clear SASD-level contracts.

Ordinary differential equations and boundary-value problems

This chapter describes the currently stable C# APIs for initial-value and boundary-value problems. It grows as the remaining V1 routines are implemented.

8.1 First-order initial-value problems

A scalar first-order problem specifies

$$y' = f(x, y) \text{ and } y(x_0) = y_0.$$

The toolkit currently offers fixed-step fourth-order Runge-Kutta (RK4), adaptive Runge-Kutta-Fehlberg 4(5) (RKF45), and a fourth-order Adams-Bashforth/Adams-Moulton predictor-corrector method.

8.2 Fixed-step RK4

Use RK4 when you deliberately want a predictable, fixed nominal step size and already know a suitable resolution.

```
using Sasd.Numerics.DifferentialEquations;

var points = RungeKutta.FourthOrder(
    (_, y) => y,
    x0: 0.0,
    y0: 1.0,
    xEnd: 1.0,
    step: 0.01);

Console.WriteLine(points[^1].Y); // approximately e
```

The solver shortens only the final step when necessary so the trajectory ends exactly at xEnd.

8.3 Second-order equations with RK4

A scalar second-order problem has the form

$$y'' = g(x, y, y')$$

and needs two initial values: $y(x_0)$ and $y'(x_0)$. You can always rewrite such a problem manually as a two-component first-order system. `FourthOrderSecondOrder` performs that standard transformation for you and returns both y and y' .

```
using Sasd.Numerics.DifferentialEquations;

// Harmonic oscillator: y'' = -y, y(0)=0, y'(0)=1.
var points = RungeKutta.FourthOrderSecondOrder(
    (_, y, _) => -y,
    x0: 0.0,
    y0: 0.0,
    firstDerivative0: 1.0,
    xEnd: Math.PI / 2.0,
    step: 0.01);

var final = points[^1];
Console.WriteLine(final.Y); // approximately 1
Console.WriteLine(final.FirstDerivative); // approximately 0
```

This is a convenience API, not a second independent RK4 implementation. Internally the equation is transformed to $y'=v$, $v'=g(x,y,v)$ and passed through the same tested system RK4 core.

8.4 *Nth-order equations with RK4*

For a scalar problem of order n , write the equation so the highest derivative is isolated:

$$y^{(n)} = g(x, y, y', \dots, y^{(n-1)}).$$

FourthOrderNthOrder accepts the lower derivatives as one ordered initial state. Element zero is y , element one is y' , and so on. The number of elements defines the equation order.

```
using Sasd.Numerics.DifferentialEquations;

// y''' = -y', with y(0)=0, y'(0)=1, y''(0)=0.
var points = RungeKutta.FourthOrderNthOrder(
    (_, state) => -state[1],
    x0: 0.0,
    initialState: [0.0, 1.0, 0.0],
    xEnd: Math.PI / 2.0,
    step: 0.01);

var final = points[^1];
Console.WriteLine(final.Y); // approximately 1
Console.WriteLine(final.GetDerivative(1)); // approximately 0
Console.WriteLine(final.GetDerivative(2)); // approximately -1
```

Each NthOrderOdePoint stores a read-only snapshot $[y, y', \dots, y^{(n-1)}]$. GetDerivative(0) is the same value as Y . The solver performs the standard companion-system transformation and then reuses the same RK4 system core; there is no separate Runge-Kutta formula for every possible order.

Use this API when the original equation is naturally written in higher-order form. If your model is already a set of interacting first-order variables, use FourthOrderSystem directly.

8.5 Coupled second-order systems with RK4

Many physical models contain several second-order variables that influence one another. Write the system as

$$Y'' = G(x, Y, Y').$$

FourthOrderSecondOrderSystem keeps values and first derivatives in separate vectors at the API boundary, while internally reducing the problem to one ordinary first-order RK4 system.

```
using Sasd.Numerics.DifferentialEquations;

var end = Math.PI / (2.0 * Math.Sqrt(2.0));
var points = RungeKutta.FourthOrderSecondOrderSystem(
    (_, values, _) =>
    {
        var difference = values[0] - values[1];
        return [-difference, difference];
    },
    x0: 0.0,
    initialValues: [1.0, -1.0],
    initialFirstDerivatives: [0.0, 0.0],
    xEnd: end,
    step: 0.01);

var final = points[^1];
Console.WriteLine(final.GetValue(0));
Console.WriteLine(final.GetFirstDerivative(0));
```

Both initial vectors must have the same dimension, and the callback must return exactly one second derivative per equation. SecondOrderSystemOdePoint copies its vectors, so later changes to caller-owned arrays cannot alter the computed trajectory.

8.6 Linear boundary-value problems with shooting

A boundary-value problem can prescribe the solution at **both** ends instead of supplying an initial derivative. The linear shooting API handles

$$y'' = p(x)y' + q(x)y + r(x)$$

with $y(x_0)=\alpha$ and $y(x_{\text{End}})=\beta$.

```
using Sasd.Numerics.DifferentialEquations;

// y'' = 2, y(0)=1, y(1)=4 -> y = 1 + 2x + x^2.
var result = LinearShooting.Solve(
    _ => 0.0,
    _ => 0.0,
    _ => 2.0,
    x0: 0.0,
    leftValue: 1.0,
    xEnd: 1.0,
    rightValue: 4.0,
    step: 0.05);

Console.WriteLine(result.InitialSlope);           // approximately 2
```

```
Console.WriteLine(result.FinalPoint.Y);           // approximately 4
Console.WriteLine(result.RightBoundaryResidual); // near zero
```

Linear shooting integrates a particular and a homogeneous sensitivity solution with RK4. Because the dependence on the unknown initial slope is linear, the final correction can be computed directly rather than by iteration.

AuxiliaryRightValue reports the sensitivity denominator. If its absolute value falls below singularityTolerance, the boundary map is treated as singular or numerically ill-conditioned instead of dividing by an unstable small number.

8.7 *Nonlinear boundary-value problems with shooting*

For a nonlinear problem

$$y'' = g(x, y, y')$$

with $y(x_0)=\alpha$ and $y(x_{\text{End}})=\beta$, the right-end value generally depends nonlinearly on the unknown initial slope. NonlinearShooting.Solve therefore tries two initial slopes and uses the secant method to drive the boundary residual

$$R(s) = y(x_{\text{End}}; s) - \beta$$

toward zero.

```
using Sasd.Numerics.DifferentialEquations;

// Exact solution y = 1/(1-x): y'' = 2y^3,
// y(0)=1 and y(0.5)=2, with exact initial slope 1.
var result = NonlinearShooting.Solve(
    (_, y, _) => 2.0 * y * y * y,
    x0: 0.0,
    leftValue: 1.0,
    xEnd: 0.5,
    rightValue: 2.0,
    step: 0.005,
    firstSlopeGuess: 0.5,
    secondSlopeGuess: 1.5,
    options: new NonlinearShootingOptions(
        BoundaryTolerance: 1e-10,
        MaximumIterations: 25));

if (result.Converged)
{
    Console.WriteLine(result.InitialSlope);           // approximately 1
    Console.WriteLine(result.FinalPoint.Y);           // approximately 2
    Console.WriteLine(result.RightBoundaryResidual); // close to zero
}
else
{
    Console.WriteLine($"{result.Status}: {result.Message}");
}
```

A nonlinear shooting failure is not automatically an invalid input. MaximumIterationsReached means that the allowed secant corrections were exhausted; NumericalBreakdown can occur when the two residuals become too similar for a stable secant update. In both cases the result still exposes the final usable trajectory for diagnosis.

The solver verifies the actual right-boundary residual before reporting Converged. This matters because a slope sequence can stagnate even while the requested endpoint is still missed.

The two starting slopes matter. Nonlinear BVPs can have multiple solutions, and different guesses can lead to different roots of the shooting residual. For important calculations, document the guesses and repeat with alternative starting values when multiple solutions are plausible.

8.8 Adaptive RKF45

Use RKF45 when the solution changes at different rates over the interval or when you prefer to state an error tolerance instead of manually choosing one fixed step size.

```
using Sasd.Numerics.DifferentialEquations;

var result = RungeKuttaFehlberg.Integrate(
    (_, y) => y,
    x0: 0.0,
    y0: 1.0,
    xEnd: 1.0,
    new RungeKuttaFehlbergOptions
    {
        InitialStep = 0.25,
        MinimumStep = 1e-8,
        MaximumStep = 0.5,
        AbsoluteTolerance = 1e-10,
        RelativeTolerance = 1e-10
    });

if (result.Completed)
{
    Console.WriteLine(result.FinalPoint.Y);
    Console.WriteLine($"Accepted: {result.AcceptedSteps}, rejected: {result.RejectedSteps}");
}
else
{
    Console.WriteLine($"Stopped with {result.Status}: {result.Message}");
}
```

The adaptive solver computes two related estimates for every trial step. Their difference estimates local error. A trial is accepted when the estimate satisfies the configured absolute-plus-relative tolerance; otherwise the trial is rejected and retried with a smaller step.

8.9 Adams predictor-corrector

The Adams method is useful when you want a regular output grid and the derivative is expensive enough that reusing earlier derivative values is attractive. The current implementation uses the fourth-order Adams-Bashforth predictor followed by the fourth-order Adams-Moulton corrector.

```
using Sasd.Numerics.DifferentialEquations;

var points = AdamsBashforthMoulton.Integrate(
    (_, y) => y,
    x0: 0.0,
    y0: 1.0,
    xEnd: 1.0,
    maximumStep: 0.1);

Console.WriteLine(points[^1].Y); // approximately e
```

A multistep formula needs history. SASD Math Toolkit therefore calculates the first three intervals with RK4 and then switches to the AB4/AM4 pair. One Adams-Moulton correction pass is the default.

8.9.1 Why maximumStep is not always the actual step

The AB4/AM4 coefficients require equal spacing. If maximumStep does not divide the interval exactly, the solver does **not** append a shortened final step. Instead it selects the smallest integer step count that respects the maximum and spreads those steps evenly over the complete interval.

8.10 Choosing between the methods

Use RK4 when a simple fixed-step reference calculation is desirable. Use RKF45 when automatic local error control and variable steps are more important. Use Adams-Bashforth/Moulton when a regular grid and derivative-history reuse fit the problem well. Use linear shooting for a linear equation with values prescribed at both endpoints. Use nonlinear shooting when the equation itself is nonlinear and the missing initial slope must be found iteratively.

None of these methods is a universal answer for stiff differential equations. If results change strongly when the step or tolerance is tightened, investigate numerical stability rather than assuming more printed digits imply more accuracy.

8.11 Boundary residual versus global accuracy

For both shooting solvers, a tiny RightBoundaryResidual only means that the final discrete trajectory lands close to the requested endpoint. It is not an error bound for the interior solution. Repeat the computation with a smaller RK4 step and compare the quantities you care about.

For nonlinear shooting, BoundaryTolerance controls the slope search while step controls RK4 discretization. Tightening only one of them does not automatically improve the other error source.

8.12 Current limits

RK4 covers scalar first-order, scalar second-order, scalar nth-order, coupled first-order and coupled second-order systems. RKF45 is currently scalar and forward-only, and Adams is scalar with fixed spacing. The historical V1 linear and nonlinear shooting routines for scalar second-order Dirichlet boundary-value problems are now both implemented. More general Neumann/Robin conditions, multiple shooting, continuation methods and dedicated stiff BVP solvers remain outside the current V1 compatibility target.

Least-squares approximation

Least-squares methods fit a model to observations that cannot usually be matched exactly. SASD Math Toolkit supports polynomial fitting and arbitrary models that are linear combinations of caller-supplied basis functions, plus the named curve models retained from the classical compatibility foundation.

9.1 Polynomial fitting

For a polynomial

$$y = c_0 + c_1x + \dots + c_nx^n$$

use `FitPolynomial` and choose the degree explicitly.

```
using Sasd.Numerics.Approximation;

double[] x = [-2, -1, 0, 1, 2];
double[] y = [17, 6, 1, 2, 9];

var coefficients = LeastSquares.FitPolynomial(x, y, degree: 2);
var predicted = LeastSquares.EvaluatePolynomial(coefficients, 1.5);
```

Coefficients are returned in ascending power order: c_0 , c_1 , c_2 , and so on. The polynomial convenience API still requires at least degree + 1 samples; it does not silently choose an underdetermined coefficient vector.

9.2 Power-law fitting

A power law has the form $y = a * x^b$. `FitPowerLaw` applies the logarithmic transformation $\ln(y) = \ln(a) + b*\ln(x)$, so both x and y must be strictly positive.

9.3 Exponential fitting

`FitExponential` fits $y = a * \exp(b*x)$. x may be any finite real value, but y must be positive because the implementation fits $\ln(y) = \ln(a) + b*x$.

9.4 Logarithmic fitting

`FitLogarithmic` fits $y = a + b*\ln(x)$. x must be positive while y may be any finite real value.

9.5 *Five-term Fourier fitting*

For periodic data with a known fundamental period or angular frequency, the five-term model is

$$a_0 + a_1 \cos(w \cdot x) + b_1 \sin(w \cdot x) + a_2 \cos(2 \cdot w \cdot x) + b_2 \sin(2 \cdot w \cdot x).$$

The frequency is supplied by the caller; this routine does not estimate it. This fit is not an FFT: it estimates a small periodic model and can use non-uniformly spaced samples.

The named Fourier helper requires a full-rank phase design because all five named coefficients are expected to be individually identifiable. Degenerate phase selections are therefore rejected even though the lower-level SVD could mathematically return one minimum-norm coefficient vector.

9.6 *Arbitrary linear basis functions*

If a model can be written as $c_0 \cdot f_0(x) + c_1 \cdot f_1(x) + \dots$, `FitBasis` fits it directly. Each basis function is evaluated exactly once per input sample and cached in the design matrix.

The general basis API now supports both the ordinary full-rank case and difficult rank structures:

- square/tall full-column-rank design -> Householder QR;
- rank-deficient or underdetermined design -> SVD minimum-norm solution.

This means `FitBasis` can be used as a lower-level regression building block even when coefficient vectors are not unique.

9.7 *Why QR remains the preferred normal case*

The original compatibility implementation used normal equations:

$$(A^T A)c = A^T y.$$

Explicitly forming $A^T A$ squares the condition number and can turn a moderately difficult regression problem into a much less reliable one.

For full-column-rank square/tall designs, the modern implementation factorizes the design matrix directly as $A = QR$ using Householder reflections. It then solves the triangular problem after applying Q^T to the observations. QR is cheaper than a complete SVD and remains the preferred default when rank is not problematic.

`QrFactorization` is public in `Sasd.Numerics.LinearAlgebra` when an application needs the decomposition or wants to reuse it for several right-hand sides.

9.8 *When SVD takes over*

When QR reports rank deficiency, or when the design matrix is wide/underdetermined, the general basis engine uses `SingularValueDecomposition`.

SVD supplies singular values and an explicit tolerance-dependent numerical rank. Components at or below the configured rank threshold are truncated, and the returned coefficient vector is the minimum-Euclidean-norm solution for the retained directions.

That distinction matters. A rank-deficient model does not have a unique coefficient vector even when all predictions are perfectly determined. SVD makes the choice explicit instead of pretending that an arbitrary coefficient set is unique.

For direct access:

```
var svd = SingularValueDecomposition.Decompose(design);
Console.WriteLine(svd.EstimatedRank);
Console.WriteLine(svd.ConditionNumber);

var coefficients = svd.SolveLeastSquares(observations);
```

See the modern dense linear algebra chapter for pseudoinverse and condition-number details.

9.9 Understanding residual diagnostics and transformations

Power-law and exponential fitting transform y before fitting and therefore minimize squared residuals in logarithmic y coordinates. Their reported RSS and RMSE are calculated afterwards in the original y domain.

Logarithmic and five-term Fourier fitting leave y unchanged. Their `ResidualSumOfSquares` is therefore directly the ordinary least-squares objective in the original y units.

9.10 Practical checks

Always inspect or plot residuals instead of relying only on fitted parameters. Scaling still matters with QR and SVD, and a numerically successful fit is not proof that the chosen model is scientifically appropriate.

For rank-sensitive work, inspect singular values and rank rather than treating a returned coefficient vector as proof that every parameter is independently measurable. The historical model set remains available, while further development now targets regression diagnostics, weighting, robust methods and the wider requirements of SASD Statistical Workbench.

FFT, convolution and correlation

The Fast Fourier Transform (FFT) changes a sampled sequence from the time/sample domain into a discrete frequency representation. SASD Math Toolkit currently uses a readable radix-2 Cooley-Tukey reference implementation.

10.1 Complex FFT

For complex samples, use Forward and Inverse:

```
using System.Numerics;
using Sasd.Numerics.Transforms;

Complex[] samples = [1, 2, 3, 4];
var spectrum = FastFourierTransform.Forward(samples);
var restored = FastFourierTransform.Inverse(spectrum);
```

The forward transform is unscaled. The inverse transform divides by the sequence length, so a forward/inverse round trip returns the original sequence within floating-point accuracy.

Every non-empty sequence must currently have a power-of-two length.

10.2 Real input: full or compact spectrum

ForwardReal is the straightforward compatibility API. It converts the real samples to complex values and returns all N Fourier bins.

For most application code, prefer ForwardRealCompact:

```
using Sasd.Numerics.Transforms;

double[] samples = [1.0, 0.0, -1.0, 0.0, 1.0, 0.0, -1.0, 0.0];
var spectrum = FastFourierTransform.ForwardRealCompact(samples);

Console.WriteLine(spectrum.OriginalLength); // 8
Console.WriteLine(spectrum.BinCount);      // 5

for (var k = 0; k < spectrum.BinCount; k++)
{
    Console.WriteLine($"bin={k}, f/Fs={spectrum.GetNormalizedFrequency(k)},
        magnitude={spectrum[k].Magnitude}");
}
```

Because the input is real, the negative-frequency bins are complex conjugates of the positive-frequency bins. Storing both halves would duplicate information.

RealFftSpectrum therefore stores DC through Nyquist only.

If your sample rate is sampleRate, the physical frequency of bin k is

```
var frequencyHz = spectrum.GetNormalizedFrequency(k) * sampleRate;
```

10.3 Reconstructing the real signal

Use InverseReal with the compact result:

```
var restored = FastFourierTransform.InverseReal(spectrum);
```

The toolkit rebuilds the omitted conjugate half internally. Application code does not need to know the packing layout.

10.4 FFT versus Fourier least-squares approximation

Do not confuse this chapter with LeastSquares.FitFiveTermFourier. The FFT analyzes frequency bins of a regularly sampled sequence. The five-term least-squares model fits a small periodic model at a caller-specified fundamental frequency and can work with irregular x coordinates. They solve different problems despite both using sine/cosine ideas.

10.5 Linear convolution

Convolution combines two finite sequences, for example a signal and an impulse response. Both real and complex overloads return the full linear convolution:

```
using System.Numerics;
using Sasd.Numerics.Transforms;

double[] signal = [1.0, 2.0, 1.0];
double[] kernel = [0.5, -0.5];
var filtered = FastFourierTransform.ConvolveReal(signal, kernel);

Complex[] complexSignal = [new(1, 1), new(2, -1)];
Complex[] complexKernel = [new(3, 0), new(0, -1)];
var complexFiltered = FastFourierTransform.ConvolveComplex(complexSignal,
    complexKernel);
```

For non-empty inputs the result contains left.Count + right.Count - 1 samples. You do not need to pad inputs to powers of two yourself; the helper chooses an internal FFT length and removes the padding afterwards.

10.6 Cross-correlation and lag ordering

Cross-correlation measures similarity while one sequence is shifted relative to another. Use CrossCorrelateReal or CrossCorrelateComplex.

For complex samples the toolkit uses conjugation:

```
r[l] = sum_k left[k] * conjugate(right[k-l]).
```

The returned array is ordered from lag $-(\text{right.Count} - 1)$ through $\text{left.Count} - 1$. If i is a result index, convert it to lag with

```
var lag = i - (right.Count - 1);
```

The zero-lag value is therefore found at index $\text{right.Count} - 1$. For complex data it is the overlapping complex inner product.

```
var correlation = FastFourierTransform.CrossCorrelateComplex(complexSignal,  
    complexKernel);  
var zeroLag = correlation[complexKernel.Length - 1];
```

Correlation conventions differ between libraries, so keep this index-to-lag rule with application code that interprets peak positions.

10.7 Empty and invalid inputs

Convolution or correlation with an empty sequence returns an empty result. Real and complex transform helpers reject NaN and infinity. For direct FFT calls, non-empty input lengths must still be powers of two.

10.8 Practical limits

The current implementation prioritizes clarity and testability rather than maximum FFT throughput. It allocates intermediate arrays; compact real FFT currently computes a full transform before dropping the redundant half, and convolution always uses the FFT even when a tiny direct convolution might be faster. Later optimized or native backends can improve these details while preserving the public API.

Always be explicit about sample rate, windowing, scaling and spectral leakage when interpreting a spectrum. The FFT computes the transform of the samples you provide; it does not automatically apply a window or infer a physical sampling interval.

Numerical recipes, diagnostics and common failure modes

A numerical routine rarely returns only a number. A trustworthy application also needs to know **how the algorithm stopped, how well the returned value satisfies the mathematical problem, which tolerance was used, and whether the problem itself may be ill-conditioned.**

This chapter brings the diagnostic conventions of the SASD Math Toolkit together in one place. It does not replace the domain chapters. Instead, it provides a common way to read results from root finding, integration, linear algebra, eigenvalue algorithms, ODE solvers and approximation routines.

11.1 1. Four questions before accepting a numerical result

Before treating a returned value as usable, ask four separate questions:

1. **Was the call valid?** Invalid dimensions, non-finite input, impossible options or an invalid interval are input-contract problems and normally raise a .NET exception.
2. **How did the algorithm terminate?** Iterative and adaptive routines expose a status such as Converged, MaximumIterationsReached, NumericalBreakdown, NotBracketed, MaximumDepthReached or MinimumStepSizeReached.
3. **How large is the defect?** A residual or error estimate says something about the returned approximation, but its meaning is specific to the problem.
4. **Is the problem well-conditioned enough for the requested interpretation?**
A tiny residual does not automatically imply a tiny error in the unknown.

These questions are deliberately separate in the API. A status is not a residual, a residual is not a forward error, and a tolerance is not a guarantee of significant digits.

11.2 2. Exceptions and result statuses mean different things

The C#/.NET implementation follows a simple rule:

- **Invalid API usage or invalid numerical input** is rejected with normal .NET exceptions such as `ArgumentException` or `ArgumentOutOfRangeException`.
- **Expected numerical outcomes after a valid call** are normally reported through result status objects.
- **Direct arithmetic that cannot produce a meaningful finite result** may raise `ArithmeticException` when there is no useful iterative status to return.

For example, a bisection interval whose endpoints do not bracket a sign change

is a mathematically meaningful outcome. `RootResult.Status` therefore reports `IterationStatus.NotBracketed`. By contrast, passing `double.NaN` as an input is an invalid call and is rejected before iteration starts.

This separation matters in application code. Do not use exceptions as the normal branch for “the method did not converge”, and do not ignore exceptions as if they were merely another convergence status.

11.3 3. Converged is a termination statement, not a proof of exactness

The generic `IterativeResult<T>` contains a `Status`, an `Iterations` count, a `Value`, an optional `Residual` and an optional diagnostic `Message`.

```
var result = LinearSystemSolvers.GaussSeidel(  
    matrix,  
    rightHandSide,  
    tolerance: 1e-10,  
    maximumIterations: 500);  
  
if (!result.Converged)  
{  
    Console.WriteLine($"Stopped with {result.Status}: {result.Message}");  
}  
  
if (result.HasFiniteResidual)  
{  
    Console.WriteLine($"Residual = {result.Residual:E3}");  
}
```

Converged means that the algorithm satisfied **its documented criterion**. It does not mean that the returned mathematical quantity is exact. A converged result can still be affected by conditioning, floating-point rounding, model error, discretization error or an inappropriate problem scale.

The reverse is also important: a non-converged result may still contain a useful best-so-far approximation and a finite residual. The new `HasFiniteResidual` property is intentionally independent of `Converged`.

11.4 4. Residual, forward error and estimated error are not interchangeable

11.4.1 Root finding

For a scalar root estimate `x`, `RootResult.Residual` is

$|f(x)|$.

A small value shows that the returned `x` nearly satisfies the equation $f(x)=0$. It does **not** directly measure $|x-x^*|$, where x^* is the exact root. Near a multiple root or where f' is very small, a small function residual can coexist with a much larger error in `x`.

```
var root = RootSolvers.NewtonRaphson(  
    x => Math.Cos(x) - x,
```

```

x => -Math.Sin(x) - 1.0,
0.5);

Console.WriteLine(root.Status);
Console.WriteLine(root.Root);
Console.WriteLine(root.Residual);

```

11.4.2 Linear systems

For $Ax=b$, `LinearSystemSolvers.ResidualInfinityNorm` computes

$\|Ax-b\|_\infty$.

That is an equation defect, not a direct bound on $\|x-x^*\|$. If A is ill-conditioned, a small residual may still correspond to a sensitive solution.

```

var x = LinearSystemSolvers.SolveGaussian(matrix, rightHandSide);
var residual = LinearSystemSolvers.ResidualInfinityNorm(matrix, x, rightHandSide
);

```

For direct solvers, computing this residual after the solve is a useful independent application-level check because the direct method itself does not return an iterative result envelope.

11.4.3 Eigenpairs

For $Av=\lambda v$, `EigenSolvers.EigenpairResidualNorm` computes

$\|Av-\lambda v\|_2$.

```

var result = EigenSolvers.PowerMethod(matrix);
var residual = EigenSolvers.EigenpairResidualNorm(matrix, result.Value);

```

A small eigenpair residual verifies the eigen-equation well in an absolute sense. It does not say that the eigenvalue is insensitive to perturbations, and repeated or tightly clustered eigenvalues can make individual eigenvectors intrinsically sensitive.

11.4.4 Adaptive integration

`AdaptiveIntegrationResult.EstimatedError` and `RombergIntegrationResult.EstimatedError` are **algorithmic error indicators** derived from refinement or extrapolation. They are not comparisons with an exact integral that the library somehow knows.

The estimate is useful for controlling refinement and for diagnostics. When an independent analytic result or a much higher-accuracy reference is available, compare against that as a separate validation step.

11.4.5 Adaptive ODE integration

Runge-Kutta-Fehlberg controls a **local step error estimate**. Successful local error control does not turn the accumulated trajectory into an exact global solution. `AdaptiveOdeResult` therefore reports whether the end point was reached and how many trial steps were accepted or rejected; application validation may still compare selected points with invariants, analytical solutions or a finer independent run.

11.5 5. Tolerance must be interpreted in the scale of the problem

A tolerance such as $1e-10$ has no universal meaning by itself.

Suppose one problem naturally contains values around $1e-6$ while another contains values around $1e12$. An absolute threshold of $1e-10$ is extremely strict for one scale and potentially meaningless for another. Some toolkit algorithms use absolute tolerances because that is the clearest historical/reference contract; adaptive RKF has explicit absolute and relative tolerance controls where both are important.

The practical rule is:

- read the method documentation to learn **what quantity is compared with the tolerance**;
- choose tolerances from the physical or mathematical scale of the problem;
- do not interpret `NumericConstants.DefaultTolerance` as a promise of a fixed number of correct decimal places;
- do not use `NumericConstants.NearlyZero` as a universal floating-point equality test.

`NearlyZero` is an internal/reference guard threshold for pivots, divisors and norms that are numerically negligible at the scale assumed by a particular routine. Public APIs such as LU factorization and inverse iteration expose a pivot tolerance when callers need control over that decision.

11.6 6. Common termination statuses and what to do next

11.6.1 `MaximumIterationsReached`

The algorithm remained numerically meaningful but did not satisfy its criterion within the configured limit.

Useful next questions are: Is the tolerance realistic? Is the initial guess poor? Is convergence intrinsically slow? Does the method fit the problem? Is the residual still decreasing?

Increasing the iteration limit can be reasonable only after answering those questions. A larger limit is not a substitute for a suitable algorithm.

11.6.2 `NumericalBreakdown`

A required numerical invariant failed: a divisor became negligible, an iterate became non-finite, overflow occurred, or continuation would otherwise be meaningless.

Do not automatically retry with a much looser tolerance. Inspect the diagnostic message and the problem scaling first. A breakdown often points to a structural issue rather than simple lack of patience.

11.6.3 `NotBracketed`

A bracketing root method was given an interval without the required sign change. Choose a better interval or use problem knowledge to find a bracket. Do not reinterpret this as “the function has no root anywhere”.

11.6.4 MaximumDepthReached / NumericalResolutionReached

Adaptive quadrature could not continue normal refinement. The returned estimate is the best value available at the limit, but Converged is false. Singularities, discontinuities, extreme oscillation or an unrealistic tolerance may be responsible.

11.6.5 MinimumStepSizeReached

Adaptive ODE integration would need a step smaller than the configured minimum to satisfy the requested local error tolerance. Review the tolerance, minimum step and the behavior of the differential equation. This can also indicate stiffness, which RKF45 is not designed to solve efficiently.

11.6.6 Singular or numerically singular matrix

Direct linear algebra routines may throw ArithmeticException when pivots fall below the configured threshold. Distinguish three cases: a genuinely singular matrix, a severely ill-conditioned matrix, and a matrix whose scaling makes the chosen absolute pivot tolerance unsuitable.

11.7 7. *Non-finite values are treated as failures, not ordinary data*

The V1 numerical algorithms expect finite floating-point inputs unless an API explicitly says otherwise. NaN and infinity are rejected at public boundaries where practical. If a callback supplied by the application returns a non-finite value during an algorithm, the routine either reports a numerical breakdown or throws an arithmetic exception according to its contract.

This policy is intentional. Letting NaN silently propagate through a hundred iterations can produce an apparently completed workflow with no useful diagnosis.

Applications should apply the same rule to imported data: validate before numerical processing, and preserve enough context to identify the original row, sample, observation or parameter that was invalid.

11.8 8. *Conditioning is different from algorithmic convergence*

A stable algorithm cannot make an ill-conditioned problem well-conditioned.

Examples:

- nearly singular linear systems can have large changes in x for small changes in A or b ;
- multiple or clustered polynomial roots can be very sensitive to coefficient perturbations;
- repeated or clustered eigenvalues can make individual eigenvectors unstable;
- numerical differentiation amplifies noise;
- high-degree polynomial interpolation can be a poor model even when the interpo-

lation equations are solved exactly.

Therefore, “the solver converged” is only one part of the quality story. For important results, also inspect problem structure and sensitivity.

11.9 9. Independent verification recipes

For important calculations, prefer a second check that is not merely the same stopping test repeated.

11.9.1 Root

Evaluate the original function at the reported root and record `RootResult.Residual`. If a derivative is available, inspect whether the root lies in a region where the function is nearly flat.

11.9.2 Direct linear solve

Recompute $\|A \cdot x - b\|_\infty$ with `ResidualInfinityNorm`. For highly sensitive applications, compare with a scaled formulation or an independent high-quality linear algebra package.

11.9.3 Eigenpair

Use `EigenpairResidualNorm` on the original matrix. For a complete symmetric decomposition, also check orthogonality and, when appropriate, reconstruction against A .

11.9.4 Integration

Repeat with a stricter tolerance or a different method when the integrand is difficult. Agreement between two genuinely different methods is stronger evidence than merely increasing one recursion limit.

11.9.5 ODE

Compare against known invariants, an analytical solution, a finer step or an independent solver. For long integrations, inspect error growth over the trajectory rather than only the last point.

11.9.6 Least squares

Inspect residuals in the **original data domain**, not only transformed coordinates. For named transformed models, the SASD result types report diagnostics in the original y domain so application interpretation remains meaningful.

11.10 10. A practical acceptance pattern

For production-facing code, a useful pattern is:

```
var result = EigenSolvers.PowerMethod(
    matrix,
    tolerance: 1e-10,
    maximumIterations: 500);

if (!result.Converged)
{
    throw new InvalidOperationException(
        $"Eigenvalue calculation stopped with {result.Status}: {result.Message}");
}

var residual = EigenSolvers.EigenpairResidualNorm(matrix, result.Value);
if (residual > 1e-8)
{
    throw new InvalidOperationException(
        $"Eigenpair residual {residual:E3} exceeds the application acceptance limit.");
}

var eigenpair = result.Value;
```

Notice that the **algorithm tolerance** and the **application acceptance limit** are separate concepts. The library controls its numerical process; the application decides whether the result is good enough for its business, scientific or engineering purpose.

11.11 11. What should be logged

When a numerical result matters enough to reproduce later, store more than the final value. Useful metadata includes:

- algorithm name and relevant options;
- input scale or dataset identity;
- termination status;
- iteration/refinement/step counts;
- residual or estimated error when available;
- diagnostic message;
- toolkit version/commit for research or validation workflows.

Do not store only “success=true”. That discards the information most useful when two runs later disagree.

11.12 12. Compact checklist

Before accepting a result, verify:

- the input was finite and satisfied the API contract;
- the termination status is understood;
- the residual/error indicator has the expected definition and scale;
- the chosen tolerance is appropriate for the problem;

- iteration/depth/step limits did not stop the algorithm prematurely;
- no diagnostic message indicates breakdown;
- conditioning or data noise does not invalidate the interpretation;
- an independent check is used when the result is important.

This diagnostic discipline is intentionally part of the SASD Math Toolkit architecture. V1 favors explicit, inspectable numerical behavior over APIs that return a bare number and hide how it was obtained.

Modern dense linear algebra — 2026 additions

The classical matrix chapter remains valid. The modernization layer adds decompositions better matched to current workloads and keeps the choice of algorithm tied to matrix structure rather than treating every problem as generic Gaussian elimination.

12.1 Householder QR

General and polynomial least squares prefer Householder QR instead of normal equations. QR avoids forming $A^T A$, does not square the condition number merely to solve the fit, and exposes practical full-column-rank diagnostics.

```
var qr = QrFactorization.Decompose(design);
if (qr.IsFullColumnRank)
{
    var coefficients = qr.SolveLeastSquares(observations);
}
```

For ordinary square/tall full-rank regression, QR remains the preferred starting point because it is cheaper than computing a complete SVD.

12.2 Cholesky

For a symmetric positive-definite matrix:

```
var factorization = CholeskyFactorization.Decompose(a);
var x = factorization.Solve(b);
```

The factorization stores $A = L L^T$, can solve several right-hand sides without refactorization, and provides `LogDeterminant()` for scale-sensitive statistical work.

Cholesky should not be forced onto a merely symmetric or arbitrary matrix. Positive definiteness is part of the mathematical contract, not an implementation detail.

12.3 Singular value decomposition

The SVD handles the cases where rank and conditioning are themselves part of the problem:

```
var svd = SingularValueDecomposition.Decompose(a);
```

```
Console.WriteLine(svd.EstimatedRank);
Console.WriteLine(svd.ConditionNumber);

var minimumNorm = svd.SolveLeastSquares(b);
var pseudoInverse = svd.PseudoInverse();
```

For an $m \times n$ matrix the toolkit exposes thin factors satisfying

$$A = U * S * V^T$$

with $\min(m,n)$ singular values sorted from largest to smallest.

The managed reference implementation uses a one-sided Jacobi SVD. It deliberately avoids obtaining the decomposition from $A^T A$; the normal matrix would square the condition number and can erase weak singular directions that are exactly what SVD is supposed to diagnose.

12.3.1 Numerical rank is tolerance-dependent

EstimatedRank is based on

```
singularValue > LargestSingularValue * RelativeRankTolerance
```

The resulting absolute value is available as RankThreshold. This is a numerical decision. A mathematically nonzero singular value may intentionally be discarded when it is too small relative to the dominant scale to support a stable solution.

The same threshold controls SolveLeastSquares and PseudoInverse, so diagnostics and computation do not disagree about which directions are retained.

12.3.2 Rank-deficient least squares

For a rank-deficient or underdetermined problem, SolveLeastSquares returns the minimum-Euclidean-norm solution associated with the retained singular directions. This is often more meaningful than choosing an arbitrary subset of columns.

For example:

```
var a = new DenseMatrix(new double[,]
{
    { 1.0, 0.0, 1.0 },
    { 0.0, 1.0, 1.0 }
});

var svd = SingularValueDecomposition.Decompose(a);
var x = svd.SolveLeastSquares(new[] { 1.0, 1.0 });
```

The system has infinitely many exact solutions. The SVD returns the minimum-norm solution, approximately (1/3, 1/3, 2/3).

12.3.3 Pseudoinverse

PseudoInverse() forms the Moore-Penrose pseudoinverse using the same truncation threshold. It is useful when the matrix itself is genuinely required by a later algorithm.

For repeated right-hand sides, reuse the SVD and call `SolveLeastSquares` instead of materializing the pseudoinverse each time.

12.3.4 Condition number

For a numerically full-rank matrix, `ConditionNumber` reports the ratio of largest to smallest singular value and therefore the matrix 2-norm condition number. If the matrix is numerically rank deficient at the configured threshold, the reported condition number is positive infinity and `ReciprocalConditionNumber` is zero.

A large condition number means that small input perturbations can cause much larger changes in the solution. It is not a statement about whether the implementation is “good” or “bad”; it is a property of the numerical problem.

12.4 Matrix norms and a combined diagnostic report

Modern numerical work frequently needs to answer more than “did the solver return a vector?”. Matrix scale, rank and null-space dimension often determine whether a result is scientifically meaningful.

The `MatrixNorms` helper provides the most commonly useful norms:

```
var one = MatrixNorms.OneNorm(a);
var infinity = MatrixNorms.InfinityNorm(a);
var frobenius = MatrixNorms.FrobeniusNorm(a);
var spectral = MatrixNorms.SpectralNorm(a);
```

`OneNorm` is the maximum absolute column sum, `InfinityNorm` the maximum absolute row sum, `FrobeniusNorm` the Euclidean norm of all entries, and `SpectralNorm` the largest singular value. The Frobenius and absolute-sum implementations use scaled accumulation so large finite entries do not cause avoidable intermediate overflow.

When several diagnostics are needed together, prefer a single combined analysis:

```
var diagnostics = MatrixConditionDiagnostics.Analyze(a);

Console.WriteLine(diagnostics.EstimatedRank);
Console.WriteLine(diagnostics.LeftNullity);
Console.WriteLine(diagnostics.RightNullity);
Console.WriteLine(diagnostics.ConditionNumber2);
```

The combined report computes the inexpensive entry-based norms directly and reuses one SVD for spectral norm, rank and 2-norm conditioning. This avoids accidentally paying for several identical decompositions.

Left and right nullity are deliberately separate. A wide 2x3 matrix can have full rectangular rank 2 and still have right nullity 1. In an underdetermined system that right null space is the family of directions that can be added to one solution without changing the right-hand side.

A small residual and a large condition number can coexist. The residual says that the computed vector satisfies the represented equations closely; the condition number says that the represented equations themselves may be sensitive to tiny perturbations.

12.5 *Least-squares routing in the toolkit*

The general `LeastSquares.FitBasis` API now chooses its dense solver deliberately:

1. square/tall full-column-rank design: Householder QR;
2. rank-deficient or underdetermined design: SVD minimum-norm solution.

Named models can retain stricter identifiability rules. For example, the five-term Fourier helper still rejects a phase design that cannot identify all five named coefficients instead of silently returning one of infinitely many equivalent coefficient vectors.

12.6 *Method selection*

Problem structure	Preferred starting point
General square dense system	Pivoted LU
Symmetric positive-definite system	Cholesky
Square/tall full-rank least squares	Householder QR
Rank-deficient least squares	SVD
Underdetermined minimum-norm system	SVD
Matrix scale / 1-, infinity-, Frobenius norm	<code>MatrixNorms</code>
Numerical rank / 2-norm conditioning	SVD or <code>MatrixConditionDiagnostics</code>
Explicit pseudoinverse required	SVD
Very large dense production workload	Later optional BLAS/LAPACK-class backend

These methods are complementary. SVD is the most diagnostic of the three modern decompositions, but it is not the cheapest solution to every well-structured system.

12.7 *Performance perspective*

The current QR, Cholesky and SVD implementations are managed, dependency-free reference algorithms. They avoid obvious unnecessary work and dangerous numerical formulations, but they are not intended to beat tuned vendor BLAS/LAPACK on very large matrices. The SASD strategy is to keep these implementations deterministic and auditable, benchmark real workloads, and later add optional accelerated backends behind stable concepts rather than replacing the reference layer.

Sparse linear algebra – CSR, CG/PCG, GMRES and BiCGSTAB

Dense matrices reserve storage for every matrix position. That is the right representation for many small and medium problems, but it becomes wasteful when a large matrix contains only a small number of non-zero coefficients per row. `CsrMatrix` is the first SASD sparse representation for such workloads.

13.1 Creating a sparse matrix

For coordinate-style assembly:

```
using Sasd.Numerics.LinearAlgebra.Sparse;

var a = CsrMatrix.FromEntries(
    4,
    4,
    [
        new SparseMatrixEntry(0, 0, 4.0),
        new SparseMatrixEntry(0, 1, -1.0),
        new SparseMatrixEntry(1, 0, -1.0),
        new SparseMatrixEntry(1, 1, 4.0),
        new SparseMatrixEntry(1, 2, -1.0),
        new SparseMatrixEntry(2, 1, -1.0),
        new SparseMatrixEntry(2, 2, 4.0),
        new SparseMatrixEntry(2, 3, -1.0),
        new SparseMatrixEntry(3, 2, -1.0),
        new SparseMatrixEntry(3, 3, 4.0)
    ]
);
```

The matrix is immutable after construction. Entries are stored in canonical CSR form: sorted columns within each row, no duplicate coordinates and no explicit zero values.

Duplicate coordinates are added during `FromEntries`. This is useful when several local contributions target the same global matrix coefficient. If they cancel exactly, the coordinate disappears from compressed storage.

13.2 Matrix-vector multiplication

The ordinary convenience call allocates a result vector:

```
var y = a.Multiply(x);
```

Iterative algorithms should normally reuse work buffers instead:

```
var y = new double[a.Rows];  
a.Multiply(x, y);
```

The second form is used by iterative solvers so repeated matrix-vector products do not allocate a new result vector on every iteration.

13.3 Dense conversion and sparsification

`CsrMatrix.FromDense(dense)` stores every non-zero dense entry. A positive `absoluteZeroTolerance` deliberately discards smaller coefficients:

```
var sparse = CsrMatrix.FromDense(dense, absoluteZeroTolerance: 1e-10);
```

This threshold is absolute. Choosing it is part of the numerical model; do not assume one epsilon is suitable for matrices with very different physical scales.

`ToDense()` is useful for inspection, interoperability and tests, but sparse algorithms should not repeatedly convert back to dense storage.

13.4 Transpose

`Transpose()` returns canonical CSR for the transposed matrix without first allocating a dense matrix. A dedicated CSC type may be added later when sustained column-oriented workloads justify it.

13.5 Sparse norms

`SparseMatrixNorms` provides inexpensive norms directly from compressed storage:

```
var one = SparseMatrixNorms.OneNorm(a);  
var infinity = SparseMatrixNorms.InfinityNorm(a);  
var frobenius = SparseMatrixNorms.FrobeniusNorm(a);
```

The Frobenius implementation is scale-aware, so large finite entries do not overflow merely because an intermediate implementation squared them naively.

13.6 Conjugate Gradient for SPD systems

`ConjugateGradientSolver` solves sparse systems whose matrix is **symmetric positive definite**. It is not a generic replacement for LU or a nonsymmetric Krylov solver.

```
var result = ConjugateGradientSolver.Solve(  
    a,  
    rightHandSide,  
    options: new SparseIterativeSolverOptions  
    {  
        AbsoluteTolerance = 1e-12,  
        RelativeTolerance = 1e-10,  
    })
```

```
MaximumIterations = 1000
});
```

The shared sparse stopping rule is

```
||b - A*x||2 <= max(AbsoluteTolerance, RelativeTolerance * ||b||2)
```

Symmetry is checked by default. A strictly positive diagonal is also required because it is necessary for positive definiteness. These inexpensive checks cannot prove that an arbitrary sparse matrix is SPD; CG therefore additionally reports `NumericalBreakdown` if the search-direction curvature $p^T A p$ becomes non-positive or non-finite.

13.6.1 True residual verification

CG updates its residual recursively for efficiency. Floating-point roundoff can make that recursive residual differ from the real $b - A*x$. Before returning `Converged`, the SASD implementation therefore recomputes the true residual. If it is still above the requested threshold, CG restarts from the refreshed residual instead of reporting false convergence.

13.7 Preconditioned Conjugate Gradient

Poor scaling or an unfavorable spectrum can make CG require many iterations even when the matrix is SPD. A preconditioner applies an approximate inverse operation

```
z = M^-1 * r
```

so the Krylov iteration sees a numerically easier problem.

The public `ISparsePreconditioner` interface is solver-neutral. The first implementation is `JacobiPreconditioner`, which uses the inverse matrix diagonal:

```
var jacobi = JacobiPreconditioner.Create(a);
var result = ConjugateGradientSolver.SolvePreconditioned(
    a,
    rightHandSide,
    jacobi,
    options: new SparseIterativeSolverOptions
    {
        AbsoluteTolerance = 1e-12,
        RelativeTolerance = 1e-10,
        MaximumIterations = 1000
    });
```

Jacobi is cheap to build and apply, creates no per-application arrays, and is especially useful as a scaling baseline. It is not guaranteed to reduce the iteration count for every problem; preconditioning quality is problem dependent.

For PCG, the preconditioner must itself be SPD. `JacobiPreconditioner` has that property when built from an SPD matrix. A custom preconditioner that makes $r^T M^{-1} r$ non-positive or non-finite causes `NumericalBreakdown` rather than silent continuation with an invalid recurrence.

13.8 Restarted GMRES for general square systems

GmresSolver is the general nonsymmetric counterpart to CG/PCG. It does not require symmetry or positive definiteness and minimizes the residual over a Krylov subspace generated by the Arnoldi process.

```
var result = GmresSolver.Solve(  
    matrix,  
    rightHandSide,  
    restartLength: 30,  
    options: new SparseIterativeSolverOptions  
    {  
        AbsoluteTolerance = 1e-12,  
        RelativeTolerance = 1e-10,  
        MaximumIterations = 1000  
    });
```

Full GMRES stores one additional basis vector per iteration. restartLength bounds that memory growth. At a restart boundary the current least-squares correction is applied, the true residual is recomputed and the next Krylov cycle begins from that residual. A small restart length saves memory but can slow or stall convergence; a larger value preserves more Krylov information.

The Arnoldi basis uses two modified-Gram-Schmidt passes to reduce avoidable loss of orthogonality. Givens rotations update the small upper-Hessenberg least-squares system incrementally. The projected residual is used as an inexpensive trigger, but Converged is returned only after explicitly recomputing the true $b - A \cdot x$ residual.

13.8.1 Right-preconditioned GMRES

The same solver-neutral preconditioner abstraction is reused through SolvePreconditioned. GMRES applies it on the right:

```
A * M^-1 * y = b
```

This keeps the minimized and reported residual in the original system. Unlike PCG, GMRES does not require the preconditioner to be SPD.

```
var jacobi = JacobiPreconditioner.Create(matrix);  
var result = GmresSolver.SolvePreconditioned(  
    matrix,  
    rightHandSide,  
    jacobi,  
    restartLength: 30);
```

A terminated Arnoldi expansion is not automatically called success. If the Krylov space cannot expand and the explicitly recomputed residual is still above tolerance, the result is NumericalBreakdown with the best available solution.

13.9 BiCGSTAB for bounded-memory nonsymmetric solves

BiCgStabSolver is another general nonsymmetric Krylov solver, but unlike GMRES it uses a short recurrence and therefore keeps a fixed number of $O(n)$ work vectors.

This can be attractive when restart-basis memory matters.

```
var result = BiCgStabSolver.Solve(
    matrix,
    rightHandSide,
    options: new SparseIterativeSolverOptions
    {
        AbsoluteTolerance = 1e-12,
        RelativeTolerance = 1e-10,
        MaximumIterations = 1000
    });
```

BiCGSTAB does not directly minimize a growing least-squares problem. Its residual may therefore be less smooth than GMRES, and its short recurrence contains scalar breakdown points. A zero/non-finite shadow-residual product, alpha denominator or stabilization coefficient is reported as `NumericalBreakdown` rather than being confused with an iteration limit.

13.9.1 Right-preconditioned BiCGSTAB

BiCGSTAB reuses `ISparsePreconditioner` and applies M^{-1} to its search directions before multiplying by A :

```
var jacobi = JacobiPreconditioner.Create(matrix);
var result = BiCgStabSolver.SolvePreconditioned(
    matrix,
    rightHandSide,
    jacobi);
```

As with GMRES, the preconditioner need not be SPD. The solver still reports convergence using the true residual of the original equation. If a recursive residual first appears converged but explicit recomputation disagrees, BiCGSTAB restarts from the refreshed true residual.

13.9.2 Choosing between the sparse solvers

A useful first decision rule is:

- **CG/PCG** for SPD systems;
- **restarted GMRES** for general nonsymmetric systems when robust residual minimization is the priority;
- **BiCGSTAB** for general nonsymmetric systems when bounded basis memory is especially important and its convergence behavior is acceptable for the problem.

Do not choose only by iteration count. Memory, matvec cost, preconditioner quality and the reported true residual all matter.

13.9.3 Solver diagnostics

`SparseLinearSolveResult` is shared by CG, PCG, GMRES and BiCGSTAB. It exposes the best available solution, `IterationStatus`, completed iteration count, initial/final residual norms, the right-hand-side norm, the effective convergence threshold and the relative residual norm. `MaximumIterationsReached` therefore remains quantitatively inspectable rather than collapsing to a Boolean failure.

13.10 What sparse linear algebra does not yet provide

CSR, CG/PCG, the common preconditioner abstraction, Jacobi preconditioning, restarted GMRES and BiCGSTAB are now implemented. The next step is an architecture/reference-case consolidation round before the release candidate. More powerful preconditioners such as incomplete Cholesky/ILU and a dedicated CSC type remain deferred until concrete workloads justify them.

The distinction matters: sparse storage is a data-layout decision, solver choice depends on mathematical structure, and preconditioner choice depends on both the solver contract and the matrix.

Historical context and current V1 coverage

The C# edition deliberately follows the major numerical problem classes of the historical Numerical Methods Toolbox at a high level. It is not a source-code port. Public APIs, data types, error handling, examples and diagnostics belong to SASD Math Toolkit. The 2026 modernization goes beyond that historical catalog where current SASD applications need stronger numerical foundations.

Topic	C#/.NET edition
Root finding	Bisection, Newton-Raphson, secant, Newton-Horner, Muller, Laguerre and polynomial deflation
Interpolation	Lagrange, divided differences, natural and clamped cubic splines
Differentiation / integration	Function, tabular and spline differentiation; Newton-Cotes, adaptive rules, Romberg and Gauss-Legendre
Dense matrix routines	DenseMatrix, Gaussian elimination, pivoted LU, Householder QR, Cholesky, SVD, pseudoinverse, norms, conditioning, inverse, residuals and Gauss-Seidel
Sparse matrix routines	Canonical CSR, sparse matvec/norms, Jacobi preconditioning, CG/PCG, restarted GMRES, BiCGSTAB and independent true-residual diagnostics
Eigenvalue problems	Power/inverse-power methods, Wielandt deflation and cyclic Jacobi
Differential equations	RK4 family, RKF45, Adams predictor-corrector, linear and nonlinear shooting
Least squares	Polynomial/general-basis fits and named models; Householder QR for full-column-rank problems with SVD fallback for rank-deficient or underdetermined minimum-norm problems
FFT / applications	Complex and real FFT, compact real spectrum, convolution and cross-correlation
Demonstrations	A platform-neutral deterministic HTML/SVG report plus a modern sparse release smoke solve

Building and maintaining the LaTeX edition

The LaTeX edition is intentionally generated from the existing Markdown chapters. Editorial changes should therefore be made first in `docs/user-guide/de/` or `docs/user-guide/en/`. The `LaTeX/` directory owns layout, build logic and language-specific front/back matter.

```
cd docs/user-guide/LaTeX
make de      # German PDF
make en      # English PDF
make all     # both editions
```


License and provenance

SASD Math Toolkit is licensed under MIT. Historical Borland material and historical product names remain the property of their respective rights holders. This handbook documents the independently developed SASD C#/.NET implementation and does not reproduce the historical manual text.